

**Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky**

**Vplyv normalizačných úprav na forenznú
atribúciu zdrojového kódu**

Bakalárska práca

**Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky**

**Vplyv normalizačných úprav na forenznú
atribúciu zdrojového kódu**

Bakalárska práca

Študijný program:	Kyberbezpečnosť
Študijný odbor:	Kyberbezpečnosť
Školiteľ:	Marek Horváth
Školiace pracovisko:	Katedra počítačov a informatiky (KPI)

Košice 2025

Miroslav Moravčík

Abstrakt v slovenčine

Abstrakt v slovenčine.

Kľúčové slová v slovenčine

L^AT_EX, programovanie, sadzba textu

Abstrakt v angličtine

Abstract in English.

Kľúčové slová v angličtine

L^AT_EX, programming, typesetting

Bibliografická citácia

MORAVČÍK, Miroslav. *Vplyv normalizačných úprav na foreznú atribúciu zdrojového kódu*. Košice: Technická univerzita v Košiciach, Fakulta elektrotechniky a informatiky, 2025. 43 s. Vedúci práce: Marek Horváth

Na túto stranu vložte zadávací list. To urobíte nasledovne:

1. Zadávací list naskenujte v odtieňoch sivej (čiernobielo) na 200 až 300 DPI.
2. Sken uložte do priečinku `figures/`.
3. V súbore `metadata.tex` zadajte umiestnenie súboru pomocou príkazu
`\thesisspec{figures/zadavaci.list}`

Do jednej kópie práce však nezabudnite vložiť originál zadávacieho listu!

Čestné vyhlásenie

Vyhlasujem, že som záverečnú prácu vypracoval(a) samostatne s použitím uvedenej odbornej literatúry, ako aj s využitím nástrojov generatívnej umelej inteligencie v súlade s Metodickým pokynom o záverečných a kvalifikačných prácach a Etickým kódexom študenta Technickej univerzity v Košiciach. Umelá inteligencia bola použitá ako podporný nástroj bez porušenia zásad akademickej etiky a autorskej integrity.

Podakovanie

Na tomto mieste by som sa rád poďakoval svojmu vedúcemu práce za jeho čas a odborné vedenie počas riešenia mojej záverečnej práce. Rovnako by som sa rád poďakoval svojim rodičom a priateľom za ich podporu a povzbudzovanie počas celého môjho štúdia.

Obsah

1	Úvod	1
2	Motivácia a ciele práce	3
3	Analýza normalizačných stratégií	5
3.1	Prehľad prístupov k atribúcii autorstva zdrojového kódu	5
3.2	Komplexné prehľady metód atribúcie autorstva	8
3.3	Adversariálne techniky proti atribúcii autorstva zdrojového kódu .	10
3.4	Stylometrické metódy atribúcie autorstva zdrojového kódu	12
3.5	Zhrnutie analytickej časti	14
4	Teoretické základy atribúcie zdrojového kódu	15
4.1	Príznaky pre atribúciu autorstva	16
4.1.1	Lexikálne príznaky	16
4.1.2	Syntaktické príznaky	16
4.1.3	Sémantické príznaky	17
4.2	Modely strojového učenia pre klasifikáciu	17
5	Techniky normalizácie kódu	19
5.1	Normalizácia bielych znakov a formátovania	19
5.2	Odstraňovanie a normalizácia komentárov	20
5.3	Kanonizácia názvov premenných a funkcií	20
5.4	Transformácia na abstraktný syntaktický strom (AST)	21
5.4.1	Dopad normalizácie na rôzne typy príznakov	21
6	Konceptuálny návrh riešenia	23
6.1	Architektúra systému	23
6.1.1	Tok spracovania dát (Pipeline)	24
6.2	Výber technológií	25
6.2.1	Špecifikácia modulov	25

7	Detailný návrh riešenia	27
7.1	Analýza požiadaviek a návrh prípadov použitia	27
7.2	Návrh dátového modelovania a proces získavania dát	28
7.3	Architektúra systému a procesný tok	28
7.4	Detailný návrh modulu extrakcie "odtlačku prsta"	28
7.4.1	Algoritmy pre lexikálne príznaky	29
7.4.2	Algoritmy pre syntaktické príznaky a AST	29
7.5	Návrh modelov strojového učenia pre klasifikáciu	30
7.5.1	Výber experimentálnych modelov	30
8	Implementácia a otestované riešenie	32
8.1	Použité technológie a vývojové prostredie	32
8.2	Implementácia kľúčových častí systému	33
8.3	Experimentálne testovanie a vyhodnotenie	34
9	Vyhodnotenie	35
9.1	Metriky úspešnosti	35
9.2	Metodika experimentov a opis dátovej sady	36
9.3	Výsledky a vyhodnotenie riešenia	37
10	Záver a prínosy práce	40
10.1	Prínosy práce	41
10.2	Otvorené otázky a vízia do budúcnosti	42
	Zoznam použitej literatúry	43
	Zoznam príloh	44

Zoznam obrázkov

3.1	Ukážka reprezentácie zdrojového kódu pomocou Abstraktného syntaktického stromu (AST)	6
3.2	Princíp klasifikácie pomocou algoritmu k-najbližších susedov (k-NN)	7
3.3	Taxonómia metód atribúcie autorstva podľa He et al. [He2024]	9
3.4	Model hrozieb pri atribúcii autorstva	11
3.5	Kategorizácia stylometrických znakov využívaných pri de-anonymizácii (podľa [Caliskan2015])	13
9.1	Graf vizuálne porovnávajúci narastanie dôležitých klasifikačných metrík na základe stupňa nasadenej normalizácie a dimenzionality extrakcie. Ukazuje, že implementácia syntaktickej analýzy minimalizuje rozdiely medzi Accuracy a F1 metrikou, čo svedčí o stabilite určení autorstva a modelových predikcií.	38

Zoznam tabuliek

- 9.1 Súhrnná tabuľka vyhodnocovacích metrík pre jednotlivé testované scenáre. Pri každom scenári bol vybraný model s celkovo najlepším dosiahnutým výsledkom v danej fáze experimentu a vizualizovaný jeho rast v presnosti oproti nižšej miere normalizácie. 37

1 Úvod

Autorstvo zdrojového kódu je či už v oblasti Informatiky ,alebo kybernetickej bezpečnosti stále viac sledovaným problémom. Atribúcia autorstva je proces, ktorého úlohou je z programátorského štýlu identifikovať autora známeho alebo neznámeho kódu. Táto problematika má široké uplatnenie – od plagiátorstva v školstve, cez forenznú analýzu vyšetrovania kybernetických zločinov, až po ochranu vlastníctva. Automatizované zistenie autora kódu môže byť preto nástrojom využívaným v rôznych oblastiach bezpečnosti, ale i v oblasti školskej integrity.

Pri atribúcii autorstva sa môžu využívať rozličné metódy zamerané priamo na celkovú analýzu štruktúry a samotný obsahu zdrojového kódu, jeho syntaktických vlastností, alebo hľadanie určitých programátorských návykov samotného autora. Príkladom takýchto znakov môže byť štýl formátovania daného kódu, štýl formátovania kódu autora, umiestňovanie komentárov špecifickým spôsobom, či charakteristické utriedenie jednotlivých logických blokov kódu. Z tohto môže vzniknúť špecifická „programátorská stopa“ , ktorá môže poslúžiť na porovnanie a následnú klasifikáciu nového kódu.

Problém atribúcie spočíva v citlivosti na rozdiely v povrchnej štruktúre. Väčšina programátorov koexistuje v rôznych vývojových prostrediach a nasadzuje editorové skripty (napr. automatické formátovanie), ktoré samovoľne menia lexikálnu rovinu kódu. Tieto rozdiely nie sú podstatou ich štýlu, a preto často narúšajú správnu hypotézu modelov o autorstve. Výskumnou výzvou je z tohto dôvodu nevyhnutnosť zdrojové texty vybranými spôsobmi pred samotnou analýzou normalizovať. Cieľenou úpravou sa tak zabezpečí bezproblémové fungovanie logiky atribučných modelov, ktoré prízvukujú fundamentálnu štruktúru bez ohľadu na vizuálne prevedenie kódu.

Formulácia úlohy

Hlavným cieľom tejto práce bude analyzovať dopad predspracovaní zdrojového kódu na samotnú presnosť rôznych atribučných modelov. Zameranie bude hlavne na analýzu dopad normalizačných krokov pri kódach ako sú napríklad premenovávanie premenných, odstraňovanie komentárov, alebo zjednocovanie kódu. Výstupom bude porovnanie presnosti rôznych atribučných modelov na rovnakej množine dát po aplikovaní vybraných normalizačných techník.

2 Motivácia a ciele práce

V posledných rokoch dochádza k výraznému nárastu množstva dostupného zdrojového kódu, a to v akademickom prostredí, v open-source komunitách aj v komerčnej sfére. Tento trend prináša nové výzvy súvisiace s analýzou, správou a hodnotením zdrojového kódu, medzi ktoré patrí aj potreba identifikácie jeho pôvodu a autorstva. Atribúcia autorstva zdrojového kódu predstavuje oblasť výskumu, ktorej cieľom je určiť pravdepodobného autora programového kódu na základe jeho štylistických a štrukturálnych vlastností.

Problematika atribúcie autorstva vychádza z princípov stylometrie v oblasti prirodzeného jazyka, avšak zdrojový kód má špecifické charakteristiky, ako je formálna syntax, obmedzená lexika a silná závislosť od programovacích jazykov a vývojových nástrojov. Tieto vlastnosti si vyžadujú osobitné analytické prístupy a metódy spracovania dát. Ako bolo opísané v predchádzajúcich kapitolách, možnosti využitia siahajú od bežnej kontroly akademickej integrity až po zložitú digitálnu forenznú analýzu a hľadanie páchatelov malvéru.

Presnosť atribučných metód však naprieč všetkými spomínanými oblasťami výrazne závisí od kvality vstupných dát a spôsobu ich predspracovania. Zdrojový kód často obsahuje prvky, ktoré neodrážajú individuálny štýl autora, ale sú výsledkom používaných programovacích štandardov, vývojových prostredí alebo automatických formátovačov (Prettier, Black a podobne). Z tohto dôvodu sa v procese atribúcie autorstva využíva normalizácia zdrojového kódu, ktorej cieľom je odstrániť alebo zjednotiť nepodstatné rozdiely medzi jednotlivými kódmi.

Z tohto dôvodu sa v procese atribúcie autorstva využíva normalizácia zdrojového kódu. Jej aplikácia však prináša aj potenciálne riziko, keďže príliš agresívne normalizačné kroky môžu viesť k odstráneniu informácií, ktoré sú pre programátorský štýl autora charakteristické a dôležité. Je tak nevyhnutné systematicky skúmať vplyv rôznych normalizačných stratégií na presnosť atribúcie a identifikovať postupy zaručujúce robustnosť modelov bez straty relevantných štýlových znakov.

Hlavnou motiváciou tejto práce je preto exaktne analyzovať a kvantifikovať vplyv normalizácie zdrojového kódu na proces atribúcie autorstva. Cieľom je prispieť k lepšiemu pochopeniu toho, ktoré normalizačné stratégie sú v kontexte strojového učenia najvhodnejšie a ako presne ovplyvňujú výkonnosť konkrétnych klasifikačných modelov.

3 Analýza normalizačných stratégií

Analytická časť práce sa zameriava na prehľad a analýzu existujúcich prístupov k atribúcii autorstva zdrojového kódu so zameraním na vplyv úprav a normalizácie kódu. Pozornosť je venovaná metódam založeným na štatistických, syntaktických a strojovo-učiacich prístupoch, ako aj ich správaniu pri rôznych formách modifikácií zdrojového kódu. Cieľom tejto kapitoly je identifikovať silné a slabé stránky jednotlivých prístupov a poukázať na význam predspracovania kódu v kontexte atribúcie autorstva.

3.1 Prehľad prístupov k atribúcii autorstva zdrojového kódu

Jednou zo základných a často citovaných prác v oblasti atribúcie autorstva z zdrojového kódu je štúdia Burrowsa, Uitdenbogerd a Turpina [1], ktorá sa systematicky zameriava na porovnanie rôznych techník používaných pri identifikácii autora programového kódu. Cieľom autorov bolo analyzovať, ktoré typy charakteristických znakov a ktoré klasifikačné prístupy dosahujú najvyššiu presnosť pri rozlišovaní autorov na základe ich programátorského štýlu.

Autori vychádzajú z predpokladu, že podobne ako v oblasti atribúcie autorstva prirodzeného jazyka, aj pri zdrojovom kóde existujú opakujúce sa štýlové vzory, ktoré sú pre konkrétneho autora charakteristické. Tieto vzory sa môžu prejavovať na rôznych úrovniach – od lexikálnej podoby kódu, cez jeho syntaktickú štruktúru až po vyššie úrovne organizácie programu.

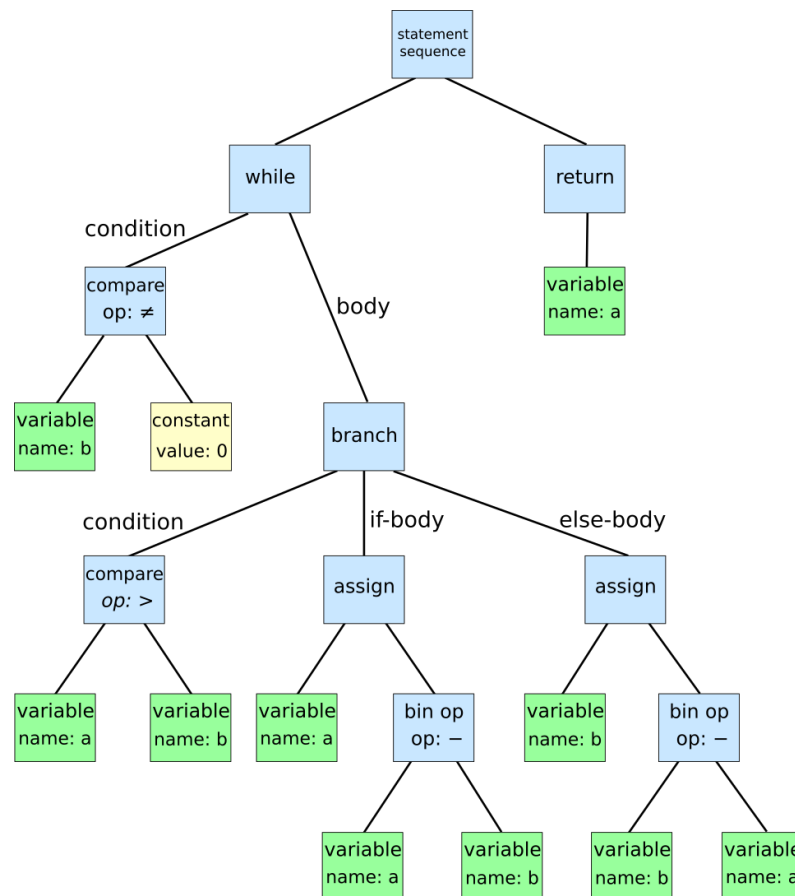
Použité typy charakteristických znakov

V rámci experimentov autori analyzovali viacero skupín črt, ktoré možno rozdeliť do troch hlavných kategórií. Prvou skupinou boli lexikálne a štatistické znaky, ktoré zahŕňali napríklad frekvenciu výskytu kľúčových slov, operátorov, identifikátorov, ako aj počet a rozloženie bielych znakov a prázdnych riadkov v kóde.

Tieto znaky sú relatívne jednoduché na extrakciu, avšak zároveň citlivé na úpravy formátovania alebo premenovanie premenných.

Druhou skupinou boli syntaktické znaky založené na analýze abstraktných syntaktických stromov (AST). Tieto znaky zachytávali štruktúru programu, napríklad typy použitých príkazov, hĺbku vnorenia riadiacich konštrukcií alebo spôsob deklarácie funkcií. Syntaktické znaky sú menej závislé od konkrétnej podoby identifikátorov, a preto sa považujú za robustnejšie voči jednoduchým modifikáciám kódu.

Treťou kategóriou boli kombinované znaky, ktoré spájali lexikálne aj syntaktické informácie. Autori predpokladali, že práve táto kombinácia môže poskytnúť najpresnejšiu reprezentáciu programátorského štýlu.



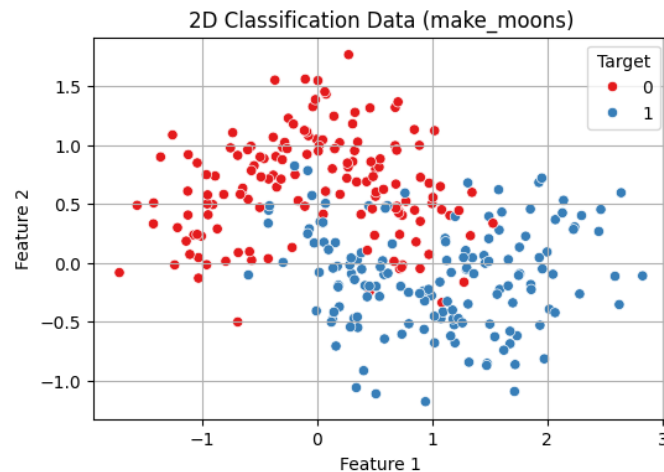
Obrázok 3.1: Ukážka reprezentácie zdrojového kódu pomocou Abstraktného syntaktického stromu (AST)

Experimentálne nastavenie a klasifikátory

Experimenty boli realizované na datasetoch obsahujúcich zdrojové kódy od viacerých autorov, pričom každý autor bol reprezentovaný viacerými programovými súborami. Autori testovali viacero klasifikačných algoritmov, medzi ktoré

patrili najmä Naive Bayes, k-nearest neighbors (k-NN) a ďalšie bežne používané štatistické klasifikátory.

Jednotlivé modely boli trénované a testované s použitím rôznych kombinácií črt, čo umožnilo objektívne porovnať ich vplyv na výslednú presnosť atribúcie autorstva. Hodnotiacim kritériom bola klasifikačná presnosť, teda podiel správne priradených autorov.



Obrázok 3.2: Princíp klasifikácie pomocou algoritmu k-najbližších susedov (k-NN)

Výsledky a porovnanie prístupov

Výsledky experimentov ukázali, že prístupy založené výlučne na lexikálnych znakoch dosahovali nižšiu presnosť, najmä v prípadoch, kde bol kód dodatočne upravovaný alebo reformátovaný. Naopak, syntaktické znaky založené na AST poskytovali stabilnejšie výsledky, keďže boli menej ovplyvnené povrchovými zmenami zdrojového kódu.

Najvyššiu presnosť však dosiahli modely využívajúce kombináciu lexikálnych a syntaktických črt. Autori zaznamenali zvýšenie presnosti o približne 10–20 % v porovnaní s modelmi založenými len na jednom type znakov. Tieto výsledky potvrdzujú hypotézu, že programátorský štýl sa prejavuje na viacerých úrovniach abstrakcie a jeho spoľahlivá identifikácia si vyžaduje komplexnejší prístup.

Význam pre ďalší výskum

Práca Burrowsa et al. poukazuje na zásadný význam výberu vhodných charakteristických znakov pri atribúcii autorstva zdrojového kódu. Zároveň nepriamo upozorňuje na problém citlivosti niektorých znakov na modifikácie kódu, ako je

zmena formátovania alebo premenovanie identifikátorov. Tento aspekt je obzvlášť dôležitý v kontexte normalizácie zdrojového kódu, keďže nevhodne zvolená normalizačná stratégia môže odstrániť práve tie znaky, ktoré sú pre autora charakteristické.

Zistenia tejto práce tvoria dôležitý teoretický základ pre ďalší výskum v oblasti atribúcie autorstva, najmä pre štúdie zamerané na porovnanie vplyvu rôznych normalizačných techník. Kombinácia viacerých typov črt sa javí ako perspektívny smer, avšak vyžaduje dôkladné zváženie spôsobu predspracovania vstupných dát.

3.2 Komplexné prehľady metód atribúcie autorstva

Významným príspevkom k systematizácii poznatkov v oblasti atribúcie autorstva je prehľadová štúdia He et al. [2], ktorá poskytuje komplexnú analýzu existujúcich prístupov, používaných charakteristických znakov, datasetov a otvorených výskumných problémov v doméne atribúcie autorstva textu aj zdrojového kódu. Na rozdiel od experimentálne orientovaných prác ide o survey článok, ktorého cieľom je zhodnotenie stavu výskumu a identifikácia hlavných trendov a limitácií súčasných metód.

Autori definujú atribúciu autorstva ako klasifikačný problém, v ktorom je neznámy artefakt (text alebo zdrojový kód) priradený jednému z množiny potenciálnych autorov na základe jeho štylistických, lexikálnych alebo štrukturálnych vlastností. Zdôrazňujú, že táto problematika má široké uplatnenie v oblasti digitálnej forenzy, detekcie plagiátorstva, kybernetickej bezpečnosti a ochrany duševného vlastníctva.

Klasifikácia prístupov k atribúcii autorstva

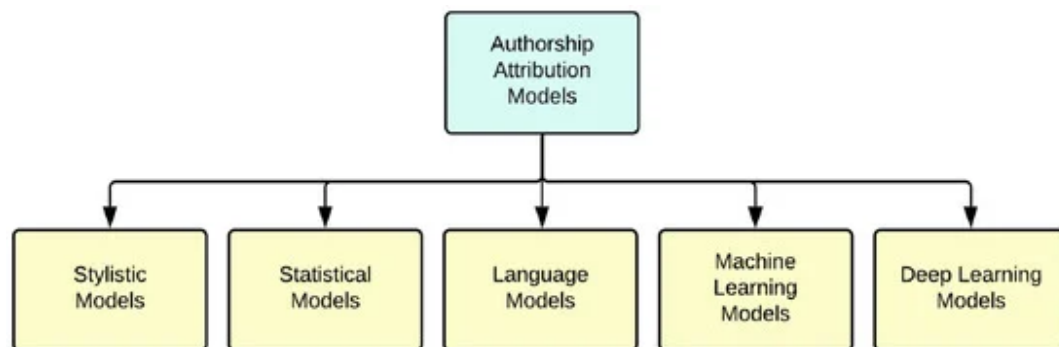
Jedným z hlavných prínosov práce je systematická taxonómia metód atribúcie autorstva, ktorú autori rozdeľujú do piatich základných kategórií. Prvou skupinou sú štylistické prístupy, ktoré sa zameriavajú na identifikáciu autorových preferencií v používaní konkrétnych jazykových alebo kódových konštrukcií. Tieto metódy vychádzajú z predpokladu, že každý autor má relatívne stabilný štýl, ktorý sa prejavuje naprieč rôznymi dielami.

Druhou kategóriou sú štatistické prístupy, ktoré využívajú kvantitatívne reprezentácie, ako sú frekvencie tokenov, n-gramy alebo pravdepodobnostné modely. Tieto metódy sú výpočtovo efektívne, avšak často trpia obmedzenou schopnosťou zachytiť hlbšie štrukturálne vlastnosti analyzovaného artefaktu.

Tretiu skupinu tvoria prístupy založené na jazykovom modelovaní, ktoré využívajú pokročilé reprezentácie textu alebo kódu prostredníctvom embeddingov a sekvenčných modelov. Tieto metódy umožňujú zachytiť komplexnejšie vzory, no zároveň vyžadujú rozsiahle tréningové dáta.

Štvrtou kategóriou sú klasické metódy strojového učenia, medzi ktoré patria napríklad Support Vector Machines, Random Forest alebo Naive Bayes klasifikátory. Tieto prístupy typicky pracujú s ručne extrahovanými črtami a ich výkon je výrazne ovplyvnený kvalitou zvolenej reprezentácie dát.

Poslednú skupinu tvoria metódy hlbokého učenia, ktoré sa snažia automaticky učiť reprezentácie relevantné pre atribúciu autorstva priamo zo vstupných dát. Autori však upozorňujú, že hoci tieto prístupy často dosahujú vysokú presnosť, ich interpretovateľnosť je obmedzená.



Obrázok 3.3: Taxonómia metód atribúcie autorstva podľa He et al. [He2024]

Typy charakteristických znakov

He et al. rozdeľujú používané charakteristické znaky do dvoch hlavných kategórií: textovo orientované a kódovo orientované črty. Textové črty zahŕňajú lexikálne, syntaktické a štylistické znaky, ako sú frekvencie slov, štruktúra viet alebo n-gramové modely. Tieto prístupy sú tradične používané v atribúcii autorstva prirodzeného jazyka.

Kódovo orientované črty sú špecifické pre zdrojový kód a zahŕňajú napríklad lexikálne znaky (použitie kľúčových slov, identifikátorov), syntaktické znaky založené na abstraktných syntaktických stromoch, ako aj grafové reprezentácie programu, ako sú riadiace alebo dátové toky. Autori poukazujú na to, že syntaktické a štrukturálne črty bývajú robustnejšie voči povrchovým úpravám kódu než čisto lexikálne znaky.

Datasey a hodnotenie

V prehľade sú analyzované najčastejšie používané datasey pre atribúciu autorstva, pričom autori upozorňujú na ich časté nedostatky, medzi ktoré patrí obmedzený počet autorov, malá rozmanitosť programovacích jazykov alebo nedostatočná reprezentácia reálnych scenárov. Na hodnotenie úspešnosti modelov sa najčastejšie používajú metriky ako presnosť, precision, recall a F1-skóre, pričom výber metriky závisí od konkrétneho typu úlohy a datasetu.

Výzvy a otvorené problémy

Jednou z hlavných výziev identifikovaných v práci je nízka robustnosť mnohých metód voči obfuskácii, refaktoringu alebo normalizácii zdrojového kódu. Autori zdôrazňujú, že povrchové úpravy kódu môžu výrazne ovplyvniť výkon modelov založených na lexikálnych črtách. Ďalším problémom je jazyková závislosť metód, ktorá obmedzuje ich prenositeľnosť medzi rôznymi programovacími jazykmi.

Práca tiež poukazuje na potrebu lepšej interpretovateľnosti modelov a štandardizovaných benchmarkov, ktoré by umožnili objektívne porovnávanie rôznych prístupov. Autori navrhujú, aby budúci výskum smeroval ku kombinácii robustných reprezentácií s vysvetliteľnými modelmi, ktoré by umožnili lepšie pochopenie rozhodovacích procesov pri atribúcii autorstva.

3.3 Adversariálne techniky proti atribúcii autorstva zdrojového kódu

Abuhamad a kol. [3] predstavil prepracovanú štúdiu, ktorá sa nezameriava primárne na zlepšenie presnosti atribúcie autorstva, ale skôr na analýzu robustnosti existujúcich atribučných metód voči adversariálnym úpravám kódu. Autori upozorňujú, že s rastúcou presnosťou atribučných modelov narastá aj riziko narušenia súkromia programátorov, ktorí chcú zostať anonymní, a preto skúmajú, ako ľahko je možné tieto metódy oklamať prostredníctvom cielenej manipulácie zdrojového kódu [3].

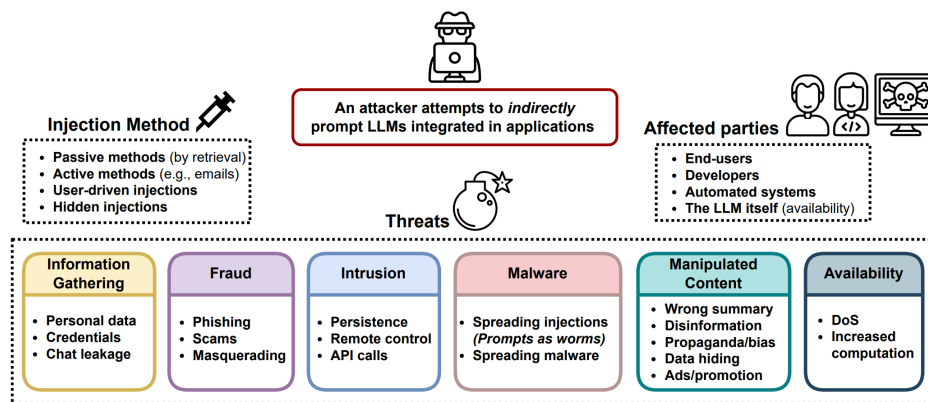
Ciele a motívácie

Hlavným cieľom tejto práce je overiť robustnosť atribučných techník voči adversariálnym útokom, ktoré môžu byť použité na: (1) degradačný útok, ktorý znižuje presnosť modelu, a (2) cielený útok, kde sa útočník snaží, aby model označil kód

za spôsobený iným autorom. Tento fokus výrazne kontrastuje s tradičným výskumom, ktorý rieši optimalizáciu klasifikátorov bez ohľadu na chytré manipulácie vstupu.

Adversariálne útoky

Autori definujú a implementujú štyri typy adversariálnych perturbácií** (kódových úprav), z ktorých dve predstavujú non-targeted útoky a dve targeted útoky. Perturbácie sú dosiahnuté pomocou adversariálnych modifikácií zdrojového kódu, ktoré nemajú meniť jeho funkčnosť, no menia štýlové a štrukturálne znaky, ktoré atribučné modely používajú na identifikáciu autora.



Obrázok 3.4: Model hrozieb pri atribúcii autorstva

Dataset a testované metódy

Experimenty boli vykonané na datase pozostávajúcom z 200 programátorov z Google Code Jam súťaže, kde každý autor bol reprezentovaný viacerými súbormi zdrojového kódu. Tento dataset je vhodný pre testovanie atribučných prístupov, pretože obsahuje širšie spektrum štýlov programovania.

Autori cielili svoje útoky na šesť štandardných atribučných metód, ktoré zahŕňali rozličné techniky extrakcie autorových charakteristík, medzi ktoré patria recurrent neural networks (RNN), convolutional neural networks (CNN) a klasické code stylometry metódy (napr. lexikálne a syntaktické znaky).

Výsledky

Experimentálne výsledky ukazujú, že existujúce atribučné metódy sú veľmi zraniteľné voči adversariálnym útokom. Pri non-targeted útokoch autori pozorujú, že:

- úspešnosť útoku (attack success rate) presahuje 98,5 %,
- identifikačná dôvera modelov klesá o viac ako 13 % v porovnaní s pôvodnými, neupravenými vstupmi.

Pri targeted útokoch, kde cieľom je, aby model označil kód ako napríklad patriaci konkrétnemu inému autorovi, sa dosahuje:

- 66 % až 88 % úspešnosť impersonácie, v závislosti od konkrétnej testovanej atribučnej techniky a nastavenia adversariálneho scenára.

Tieto výsledky naznačujú, že súčasné atribučné prístupy, vrátane metód využívajúcich hlboké neurónové siete, nemajú dostatočnú robustnosť voči cielene vytvoreným perturbáciám a môžu byť relatívne ľahko obchádzané.

Diskusia a implikácie pre výskum

Práca Abuhamad a kol. zdôrazňuje dôležitý aspekt, ktorý je často prehliadaný v tradičných prácach o atribúcii autorstva – bezpečnosť a robustnosť voči adversariálnym manipuláciám. Výsledky ukazujú, že modely, ktoré dosahujú vysokú presnosť v bežných podmienkach, môžu byť zraniteľné voči relatívne jednoduchým, no cielene navrhnutým transformáciám kódu, ktoré nemajú vplyv na jeho funkčnosť.

Tento aspekt je obzvlášť relevantný v scenároch, kde anonymita a súkromie programátorov je kľúčové, alebo kde nie je garantované, že kódové artefakty neboli modifikované škodlivým útočníkom. Práca preto otvára dôležitú diskusiu o potrebe vývoja robustnejších atribučných modelov, ktoré budú odolávať adversariálnym zásahom bez straty identifikačnej výkonnosti.

3.4 Stylometrické metódy atribúcie autorstva zdrojového kódu

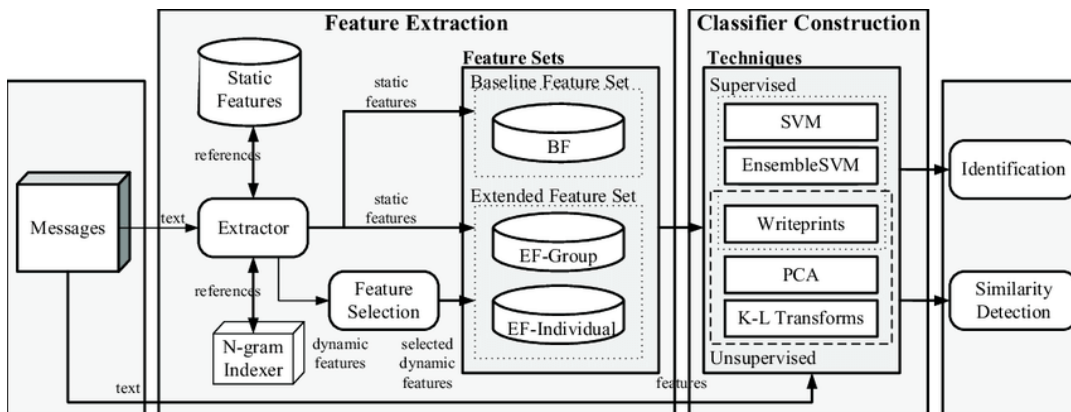
Caliskan-Islam, Bryson a Narayanan [4] predstavujú jednu z najvplyvnejších prác v oblasti atribúcie autorstva zdrojového kódu, ktorá preukazuje, že programátorský štýl je možné spoľahlivo identifikovať aj v prípadoch, kde sa autori zámerne snažia o anonymitu. Práca je významná najmä preto, že demonštruje praktické dôsledky atribúcie autorstva v oblasti bezpečnosti a súkromia.

Ciele a výskumné otázky

Hlavným cieľom práce je empiricky overiť hypotézu, že programátori zanechávajú v zdrojovom kóde dostatočne silné štylistické stopy, ktoré umožňujú ich identifikáciu aj po odstránení explicitných identifikátorov. Autori sa zameriavajú na otázku, do akej miery je možné de-anonymizovať autorov kódu použitím automatizovaných stylometrických techník.

Použité charakteristické znaky

Autori navrhujú rozsiahlu množinu črt extrahovaných zo zdrojového kódu, ktoré zahŕňajú lexikálne, syntaktické a štrukturálne znaky. Medzi analyzované črty patria napríklad frekvencie tokenov, štruktúra abstraktných syntaktických stromov, štatistiky riadiacich konštrukcií a vzory používania programovacích idiomov. Kombinácia týchto znakov umožňuje zachytiť hlbšie aspekty programátorského štýlu, ktoré sú odolnejšie voči jednoduchým úpravám kódu.



Obrázok 3.5: Kategorizácia stylometrických znakov využívaných pri de-anonymizácii (podľa [Caliskan2015])

Experimentálne nastavenie

Experimenty boli realizované na niekoľkých datasetoch vrátane verejne prístupných repozitárov a anonymných súťaží v oblasti programovania. Autori využili klasifikátory založené na Support Vector Machines a Random Forest, ktoré boli trénované na rôznych kombináciách extrahovaných črt. Výkonnosť modelov bola hodnotená pomocou presnosti a ďalších štandardných metrík.

Výsledky

Výsledky experimentov ukazujú, že navrhnuté metódy dokážu dosiahnuť veľmi vysokú presnosť atribúcie autorstva, pričom v niektorých scenároch presnosť presahuje 90 %. Autori demonštrujú, že aj po odstránení komentárov, formátovania a identifikátorov je možné autora kódu identifikovať s vysokou pravdepodobnosťou. Tieto zistenia potvrdzujú, že programátorský štýl je hlboko zakorenený v štruktúre kódu.

Diskusia a implikácie

Práca má zásadné implikácie pre oblasť softvérovej forenziky a ochrany súkromia. Autori upozorňujú, že atribúcia autorstva môže predstavovať riziko pre anonymitu programátorov, najmä v prostrediach, kde je anonymita kritická. Zároveň táto štúdia poskytuje pevný základ pre ďalší výskum zameraný na robustnosť atribučných metód a na vývoj techník na ochranu súkromia.

3.5 Zhrnutie analytickej časti

Analýza dostupnej literatúry ukazuje, že atribúcia autorstva zdrojového kódu je komplexnou úlohou, ktorej úspešnosť závisí od viacerých faktorov. Okrem samotného výberu atribučného modelu zohráva kľúčovú úlohu kvalita vstupných dát a spôsob ich predspracovania.

Normalizácia zdrojového kódu sa javí ako nevyhnutný krok na zníženie vplyvu nepodstatných úprav a zvýšenie robustnosti modelov. Zároveň však predstavuje riziko odstránenia dôležitých štýlových znakov, čo poukazuje na potrebu jej dôkladného a systematického porovnávania. Tieto zistenia vytvárajú teoretický základ pre experimentálnu časť práce, ktorá sa zameriava na vyhodnotenie účinnosti vybraných normalizačných stratégií.

4 Teoretické základy atribúcie zdrojového kódu

Atribúcia autorstva zdrojového kódu, často označovaná aj ako softvérová forenzia alebo stylometria zdrojového kódu, je proces identifikácie autora neznámeho zdrojového kódu na základe jeho programátorského štýlu. Tento štýl predstavuje súbor jedinečných návykov a preferencií programátora, ktoré sa prejavujú v spôsobe, akým píše kód. Podobne ako rukopis alebo štýl reči, aj programátorský štýl môže zanechať "odtlačok prsta", ktorý je možné analyzovať a porovnávať.

Význam tejto disciplíny neustále rastie s narastajúcim množstvom softvéru vo všetkých aspektoch moderného života. Medzi hlavné oblasti uplatnenia patrí **autorské právo a plagiátorstvo**, kde atribúcia umožňuje odhaľovanie neoprávneného kopírovania kódu v akademickom prostredí alebo v komerčných produktoch. Ďalšou dôležitou oblasťou je **kybernetická bezpečnosť**, pre ktorú je nevyhnutná identifikácia autorov malvéru alebo účastníkov kybernetických útokov. Analýza kódu tu môže pomôcť prepojiť rôzne útoky s jednou skupinou alebo konkrétnym jednotlivcom. Taktiež v rámci **forenznej analýzy** a digitálneho vyšetrovania môže atribúcia kódu poskytnúť kľúčové dôkazy o tom, kto napísal alebo neskôr modifikoval konkrétny softvér. V neposlednom rade v **softvérovom inžinierstve** pomáha analýza štýlu pri údržbe kódu, a to najmä v prípade, ak pôvodný autor už nie je k dispozícii. Navyše môže odhaliť tzv. "ghostwriting", kde jeden programátor píše kód pod menom iného.

Proces atribúcie sa zvyčajne skladá z dvoch hlavných fáz: extrakcie príznakov a klasifikácie. V prvej fáze sa zo zdrojového kódu extrahujú merateľné charakteristiky (príznačky), ktoré reprezentujú štýl autora. V druhej fáze sa tieto príznaky použijú na trénovanie modelu strojového učenia, ktorý sa naučí rozlišovať medzi štýlmi rôznych autorov a následne klasifikovať nový, neznámy kód.

4.1 Príznaky pre atribúciu autorstva

Kľúčom k úspešnej atribúcii je výber správnych príznakov, ktoré dokážu efektívne zachytiť individuálny štýl programátora. Tieto príznaky možno rozdeliť do niekoľkých kategórií.

4.1.1 Lexikálne príznaky

Lexikálne príznaky sa týkajú textovej reprezentácie kódu a nevyžadujú hĺbkovú syntaktickú analýzu. Sú relatívne jednoduché na extrakciu a zahŕňajú širokú škálu metrík. Patria sem **metriky bielych znakov**, ktoré mapujú spôsob, akým programátor používa medzery, tabulátory a nové riadky. Spadá sem priemerná dĺžka riadku, počet prázdnych riadkov, odsadenie blokov kódu (medzery vs. tabulátory) a medzery okolo operátorov i kľúčových slov. Dôležité sú aj **metriky komentárov**, určujúce hustotu a štýl komentárov. Analyzuje sa pri nich pomer komentárov ku kódu, priemerná dĺžka komentára a používanie blokových (`/* ... */`) vs. riadkových (`// ...`) komentárov. Veľký vplyv má aj **výber identifikátorov**, konkrétne dĺžka a štýl názvov premenných, funkcií a tried. Niektorí programátori preferujú krátke názvy (napr. `i`, `n`), iní deskriptívne (napr. `numberOfUsers`, `customer_id`). Sleduje sa tiež používanie konvencií ako `camelCase`, `PascalCase` alebo `snake_case`. V poslednom kroku sa vyhodnocuje **distribúcia kľúčových slov a operátorov**, čím sa myslí frekvencia výskytu kľúčových slov (napr. `for`, `while`, `if`), operátorov (`+`, `-`, `*`, `/`) a literálov (napr. číselné konštanty).

4.1.2 Syntaktické príznaky

Syntaktické príznaky idú o úroveň hlbšie a analyzujú štruktúru kódu. Na ich extrakciu je potrebné zdrojový kód parsovať a vytvoriť z neho štruktúrovanú reprezentáciu, najčastejšie **abstraktný syntaktický strom (AST)**. AST reprezentuje hierarchickú štruktúru kódu bez ohľadu na povrchové textové detaily, ako sú biele znaky alebo komentáre.

Príznaky extrahované z AST zahŕňajú základné prístupy. Prvým je **frekvencia uzlov AST**, prostredníctvom ktorej sa meria celkový počet výskytov jednotlivých typov uzlov v strome, napríklad počet cyklov (`ForStatement`, `WhileStatement`), podmienených výrazov (`IfStatement`), deklarácií funkcií a pod. Nasledujú **štrukturálne metriky AST**. Tie zohľadňujú hĺbku a šírku stromu, hĺbku vnorenia jednotlivých blokov kódu a počet potomkov pre jednotlivé typy uzlov. Tieto metriky môžu odhaliť preferenciu programátora pre zložité vnorené štruktúry alebo

jednoduchšie, ploché konštrukcie. Dôležitou súčasťou sú aj **sekvencie a n-gramy uzlov**, pre ktoré je kľúčová analýza sekvencií, v akých sa jednotlivé syntaktické konštrukcie objavujú za sebou. Preukazuje to napríklad, či programátor zvykne často inicializovať premennú ihneď po jej deklarácii.

4.1.3 Sémantické príznaky

Sémantické príznaky sa zameriavajú na význam a správanie kódu. Sú najťažšie na extrakciu, pretože vyžadujú čiastočné alebo úplné pochopenie toho, čo kód robí. Tieto príklady zahŕňajú **využitie knižníc a API**, teda preferované knižnice a funkcie, ktoré programátor používa na riešenie bežných úloh (napr. práca so súborami, sieťová komunikácia). Ďalej sem patria **algoritmické vzory**, mapujúce okrem iného používanie rekurzie vs. iterácie, a tiež výber špecifických dátových štruktúr, medzi ktoré radíme napríklad obľubu polí verzus spájaných zoznamov. V neposlednom rade plní sémantickú funkciu **riadenie toku programu**, čo zosobňuje analýza grafu toku riadenia (Control Flow Graph - CFG) zameraná na identifikáciu typických vzorov v logike programu.

4.2 Modely strojového učenia pre klasifikáciu

Po extrakcii príznakov nasleduje fáza klasifikácie, kde sa model strojového učenia trénuje na dátovej sade s kódom od známych autorov. Každý autor je reprezentovaný jednou triedou a jeho kód súborom príznakov. Natrénovaný model je potom schopný priradiť neznámy kód k najpravdepodobnejšiemu autorovi.

Medzi najčastejšie používané modely patria **Support Vector Machines (SVM)**, čo reprezentuje výkonný klasifikátor, ktorý hľadá optimálnu nadrovinu na oddelenie dátových bodov patriacich do rôznych tried. Býva tak mimoriadne efektívny vo vysokorozmerných priestoroch, čo je častý prípad pri atribúcii kódu s veľkým počtom príznakov. Uplatnenie pravidelne nachádzajú taktiež zavedené **rozhodovacie stromy a náhodné lesy (Random Forests)**. Konštatujeme tu, že rozhodovací strom vytvára potrebný model na základe série jednoduchých pravidiel (otázok) o príznakoch. Náhodný les je následne súborom viacerých rozhodovacích stromov, kde každý strom hlasuje za výslednú triedu. Táto metóda je robustná voči nadmernému prispôsobeniu (overfitting) a dokáže dobre pracovať s rôznorodými príznakmi. K tradičným osvedčeným prístupom patrí aj **naivný Bayesov klasifikátor**. Tento model je založený na Bayesovej vete a predpokladá (naivne), že príznaky sú navzájom nezávislé. Napriek tomuto zjednodušeniu je často prekvapivo efektívny, najmä pri textovej klasifikácii a teda aj pri lexikálnych

príznakoch. Moderné postupy sa však spoliehajú najmä na **neurónové siete a hlboké učenie**. Súčasné prístupy využívajú rôzne architektúry neurónových sietí, vrátane rekurentných neurónových sietí (RNN) alebo Long Short-Term Memory (LSTM) sietí, ktoré sú vhodné na spracovanie sekvenčných dát, ako je text kódu. K tomu konvolučné neurónové siete (CNN) dokážu úspešne hľadať lokálne vzory v kóde. Tieto obsiahle modely dokážu automaticky extrahovať relevantné príznaky priamo zo surového obdržaného kódu alebo z AST, čím sa zjednodušuje potreba manuálneho inžinierstva príznakov.

Voľba konkrétneho modelu a sady príznakov závisí od špecifického problému, dostupných dát a požadovanej presnosti. Kombinácia viacerých typov príznakov a použitie ansámblových metód (ako Random Forests) často vedie k najlepším výsledkom.

5 Techniky normalizácie kódu

Ako bolo spomenuté v úvode, modely na atribúciu autorstva sú často citlivé na povrchové, štrukturálne rozdiely v zdrojovom kóde, ktoré nie sú nevyhnutne súčasťou jedinečného štýlu autora. Tieto rozdiely môžu byť spôsobené použitím rôznych editorov, automatických formátovacích nástrojov (ako Prettier, Black, alebo gofmt), alebo jednoducho nedôslednosťou pri písaní kódu. Cieľom normalizácie je odstrániť tieto "šumové" variácie a zjednotiť kód do kanonickej podoby. Tým sa zabezpečí, že model strojového učenia sa bude sústrediť na hlbšie a stabilnejšie charakteristiky programátorovho štýlu.

Normalizácia je teda kľúčovým krokom predspracovania dát, ktorý predchádza extrakcii príznakov. Správne zvolené normalizačné techniky môžu výrazne zvýšiť presnosť a robustnosť atribučného modelu. Naopak, príliš agresívna normalizácia by mohla odstrániť aj cenné stylistické informácie. Nájsť správnu rovnováhu je hlavnou výzvou.

V tejto kapitole podrobne rozoberieme najbežnejšie techniky normalizácie a ich vplyv na atribúciu autorstva.

5.1 Normalizácia bielych znakov a formátovania

Spôsob, akým programátor používa biele znaky (medzery, tabulátory, nové riadky), je jedným z najzákladnejších, no zároveň najmenej spoľahlivých rysov štýlu. Je to preto, lebo automatické formátovače dokážu tieto aspekty kódu kompletne zmeniť jediným príkazom.

Techniky normalizácie formátovania zahŕňajú:

- **Zjednotenie odsadenia:** Všetky tabulátory (`\t`) sú nahradené štandardným počtom medzier (napríklad 4 medzery). Tým sa eliminuje rozdiel medzi programátormi, ktorí preferujú tabulátory, a tými, ktorí používajú medzery.
- **Odstránenie nadbytočných medzier a riadkov:** Viacnásobné medzery sú

nahradené jednou, medzery na konci riadkov sú odstránené a viacnásobné prázdne riadky sú zredukované na jeden.

- **Štandardizácia medzier okolo operátorov:** Zabezpečenie konzistentného používania medzier okolo binárnych operátorov (napr. $x = y + z$ namiesto $x=y+z$).
- **Automatické formátovanie:** Najradikálnejším prístupom je aplikácia štandardného formátovacieho nástroja na celý zdrojový kód. Tým sa úplne odstránia všetky individuálne preferencie vo formátovaní. Aj keď to eliminuje veľa šumu, zároveň to zničí všetky štylistické informácie obsiahnuté vo formátovaní.

5.2 Odstraňovanie a normalizácia komentárov

Komentáre sú ďalším zdrojom štylistických informácií, ale sú tiež náchylné na modifikácie. Programátor môže kód prevziať a pridať vlastné komentáre, alebo ich úplne odstrániť.

Možné prístupy k normalizácii komentárov:

- **Úplné odstránenie komentárov:** Táto technika odstráni všetky blokové aj riadkové komentáre zo zdrojového kódu. Je to najjednoduchší spôsob, ako eliminovať ich vplyv, ale stráca sa tým potenciálne cenný signál (napr. hustota komentárov, preferovaný typ komentárov).
- **Nahradenie placeholderom:** Namiesto úplného odstránenia môže byť každý komentár nahradený špeciálnym tokenom, napríklad `_COMMENT_`. Týmto spôsobom sa zachová informácia o polohe a počte komentárov bez toho, aby sa analyzoval ich obsah, ktorý môže byť zavádzajúci.

5.3 Kanonizácia názvov premenných a funkcií

Výber názvov pre identifikátory (premenné, funkcie, triedy) je silným indikátorom štýlu. Niektorí programátori používajú krátke, kryptické názvy, iní dlhé a deskriptívne. Táto informácia je však závislá od sémantiky problému, ktorý kód rieši. Pri porovnávaní dvoch rôznych programov od toho istého autora budú názvy premenných prirodzene odlišné.

Cieľom kanonizácie je abstrahovať od konkrétnych názvov a zamerať sa na štruktúrálny vzor ich používania.

1. **Extrakcia identifikátorov:** Najprv sa pomocou parsera identifikujú všetky používateľom definované názvy.
2. **Nahradenie štandardizovanými názvami:** Každý jedinečný názov je nahradený generickým identifikátorom. Napríklad, všetky výskyty premennej `user_name` môžu byť nahradené `VAR_1`, `user_password` nahradené `VAR_2`, funkcia `calculateTotal` nahradená `FUNC_1`, a tak ďalej.

Tento proces sa vykonáva konzistentne v rámci jedného súboru. Výsledkom je kód, kde konkrétne názvy premenných zmiznú, ale štrukturálne informácie zostanú zachované – napríklad, koľko unikátnych premenných funkcia používa, alebo ako často sa premenné opätovne používajú.

5.4 Transformácia na abstraktný syntaktický strom (AST)

Najpokročilejšou formou normalizácie je úplná abstrakcia od textovej podoby kódu a jeho transformácia na **abstraktný syntaktický strom (AST)**. Ako už bolo popísané, AST je stromová reprezentácia syntaktickej štruktúry kódu.

Výhody použitia AST ako kanonickej formy sú značné:

- **Nezávislosť od povrchových zmien:** AST je imúnny voči zmenám v bielych znakoch, komentároch a často aj vo voľbe niektorých syntaktických skratiek (napr. `x++` vs. `x = x + 1` môžu byť reprezentované rovnako).
- **Zachovanie štrukturálnej podstaty:** Reprezentuje čistú štruktúru programu a logiku, ktorú autor zvolil, bez textového balastu.

Po transformácii kódu na AST sa atribúcia ďalej vykonáva na úrovni tohto stromu. Analýza sa potom nezameriava na text, ale na štrukturálne príznaky stromu, ako sú typy a frekvencie uzlov, ich vzájomné vzťahy, hĺbka vnorenia a podobne.

Samotná serializácia AST (prevedenie stromu späť do sekvenčnej formy pre modely strojového učenia) môže byť tiež formou normalizácie. Prechádzaním stromu do hĺbky (depth-first traversal) sa získa normalizovaná sekvencia syntaktických tokenov, ktorá reprezentuje štruktúru kódu.

5.4.1 Dopad normalizácie na rôzne typy príznakov

Je dôležité si uvedomiť, ako jednotlivé normalizačné techniky ovplyvňujú rôzne kategórie príznakov:

- **Lexikálne príznaky:** Sú najviac ovplyvnené. Normalizácia formátovania a odstránenie komentárov priamo eliminujú celú triedu týchto príznakov. Kanonizácia názvov zase odstraňuje informácie o voľbe identifikátorov.
- **Syntaktické príznaky:** Sú ovplyvnené menej. Aj po agresívnej normalizácii textu zostáva štruktúra kódu, a teda aj AST, z veľkej časti zachovaná. Práve preto sú syntaktické príznaky považované za robustnejšie.

Cieľom tejto práce je empiricky zmerať, do akej miery tieto normalizačné kroky vplývajú na presnosť atribučných modelov. Je možné, že odstránením "šumových" lexikálnych príznakov sa model dokáže lepšie sústrediť na stabilnejšie syntaktické príznaky, čo v konečnom dôsledku presnosť zvýši. Alebo naopak, strata informácií môže presnosť znížiť. Odpoveď na túto otázku je kľúčová pre návrh efektívnych systémov na atribúciu autorstva.

6 Konceptuálny návrh riešenia

Na základe teoretických poznatkov z predchádzajúcich kapitol sme navrhli konceptuálny model systému na forenznú atribúciu zdrojového kódu. Cieľom tohto systému je experimentálne overiť vplyv normalizačných úprav na presnosť klasifikácie autorstva. Systém je navrhnutý modulárne, aby umožňoval jednoduché pridávanie a odoberanie jednotlivých normalizačných krokov a testovanie rôznych klasifikačných modelov.

Celý proces atribúcie v našom návrhu pozostáva z niekoľkých na seba nadväzujúcich fáz, ktoré tvoria dátovod (pipeline). Tento dátovod spracúva vstupné zdrojové kódy a na výstupe poskytuje predikciu ich autora.

6.1 Architektúra systému

Navrhovaný systém sa skladá z nasledujúcich kľúčových komponentov:

1. **Dátový modul (Dataset Manager):** Zodpovedá za načítanie a správu dátovej sady, ktorá obsahuje zdrojové kódy od viacerých autorov. Zabezpečuje rozdelenie dát na trénovaciu a testovaciu množinu.
2. **Normalizačný modul (Normalizer):** Implementuje sadu normalizačných techník, ktoré je možné flexibilne aplikovať na vstupné zdrojové kódy.
3. **Modul na extrakciu príznakov (Feature Extractor):** Transformuje normalizovaný zdrojový kód na vektor numerických alebo kategorických príznakov, ktoré slúžia ako vstup pre klasifikátor.
4. **Klasifikačný modul (Classifier):** Obsahuje implementácie rôznych modelov strojového učenia, ktoré sú trénované na extrahovaných príznakoch a následne použité na predikciu autorstva.
5. **Vyhodnocovací modul (Evaluator):** Meria a porovnáva presnosť klasifikácie pri rôznych konfiguráciách systému (t.j. s rôznymi kombináciami normalizačných krokov a modelov).

Tieto moduly sú usporiadané do logického toku spracovania, ktorý bude vizuálne znázornený pomocou diagramu architektúry v neskoršej časti práce.

6.1.1 Tok spracovania dát (Pipeline)

Proces atribúcie prebieha v dvoch hlavných režimoch: trénovacom a predikčnom.

Trénovací režim:

1. **Vstup:** Trénovacia sada zdrojových kódov s priradenými menami autorov.
2. **Krok 1: Normalizácia.** Na každý zdrojový súbor sa aplikuje zvolená sada normalizačných pravidiel (napr. odstránenie komentárov, kanonizácia premenných). Vytvára sa viacero verzií dátovej sady – jedna bez normalizácie a ďalšie pre každú kombináciu normalizačných techník.
3. **Krok 2: Extrakcia príznakov.** Z každej verzie normalizovaného kódu sa extrahujú príznaky (napr. frekvencie AST uzlov, lexikálne metriky). Výsledkom je sada vektorov príznakov pre každú verziu dát.
4. **Krok 3: Trénovanie modelu.** Pre každú sadu príznakov sa natrénuje samostatný klasifikačný model. Model sa učí mapovať vektory príznakov na konkrétnych autorov.

Predikčný (testovací) režim:

1. **Vstup:** Testovací súbor zdrojového kódu bez informácie o autorovi.
2. **Krok 1: Normalizácia.** Na súbor sa aplikujú rovnaké normalizačné pravidlá, s akými bol trénovaný daný model.
3. **Krok 2: Extrakcia príznakov.** Z normalizovaného kódu sa extrahuje vektor príznakov rovnakým spôsobom ako pri trénovaní.
4. **Krok 3: Klasifikácia.** Natrénovaný model prijme vektor príznakov a na výstupe vráti predikciu najpravdepodobnejšieho autora.
5. **Krok 4: Vyhodnotenie.** Predikcia sa porovná so skutočným autorom testovacieho súboru. Tento proces sa opakuje pre všetky súbory v testovacej sade a vypočíta sa celková presnosť modelu.

Tento systematický prístup umožní presne zmerať, ako každá normalizačná technika (alebo ich kombinácia) ovplyvňuje výslednú presnosť.

6.2 Výber technológií

Pre implementáciu prototypu navrhujeme použiť programovací jazyk **Python** z nasledujúcich dôvodov:

- **Bohatý ekosystém knižníc:** Python ponúka špičkové knižnice pre dátovú vedu a strojové učenie, ako sú `scikit-learn`, `pandas` a `numpy`.
- **Spracovanie a parsovanie kódu:** Knižnica `ast` v štandardnej knižnici Pythonu umožňuje jednoduché parsovanie Python kódu a prácu s jeho abstraktným syntaktickým stromom. Pre analýzu iných jazykov (napr. C, Java) je možné využiť externé knižnice ako `pycparser` alebo `tree-sitter`. Vzhľadom na zameranie práce bude prototyp primárne cielený na analýzu kódu v jazyku Python, čo zjednoduší implementáciu.
- **Rýchle prototypovanie:** Python je interpretovaný jazyk, ktorý umožňuje rýchly vývoj a experimentovanie, čo je ideálne pre akademický prototyp.

6.2.1 Špecifikácia modulov

Normalizačný modul Bude obsahovať sadu funkcií, kde každá implementuje jednu normalizačnú techniku:

- `remove_comments(code: str) → str`: Odstráni komentáre.
- `normalize_whitespace(code: str) → str`: Zjednotí formátovanie bielych znakov.
- `rename_variables(code: str) → str`: Prevedie kanonizáciu názvov premenných a funkcií pomocou analýzy AST.

Tieto funkcie bude možné ľubovoľne kombinovať a vytvárať tak rôzne úrovne normalizácie.

Modul na extrakciu príznakov Bude zodpovedný za transformáciu textu kódu na číselný vektor. Navrhujeme implementovať dva typy extraktorov:

- **Lexikálny extraktor:** Vypočíta metriky ako dĺžka kódu, počet riadkov, frekvencia kľúčových slov (TF-IDF).
- **Syntaktický extraktor:** Vygeneruje AST a vypočíta frekvenciu výskytu jednotlivých typov uzlov (napr. `ast.For`, `ast.If`, `ast.FunctionDef`).

Klasifikačný modul Bude využívať knižnicu `scikit-learn` na implementáciu a tréovanie nasledujúcich modelov:

- **Support Vector Machine (SVM)**
- **Random Forest Classifier**
- **Multinomial Naive Bayes**

Tieto modely reprezentujú rôzne prístupy ku klasifikácii a poskytnú komplexný pohľad na vplyv normalizácie.

Týmto návrhom sme položili základy pre implementáciu experimentálneho systému, ktorý umožní systematicky preskúmať a kvantifikovať vplyv normalizačných úprav na forenznú atribúciu zdrojového kódu, čo je hlavným cieľom tejto práce.

7 Detailný návrh riešenia

Táto kapitola predstavuje komplexný a detailný inžiniersky návrh systému na atribúciu zdrojového kódu. Kým predchádzajúca časť priblížila konceptuálny pohľad na architektúru, táto kapitola ide do hĺbky samotnej realizácie. Definuje presné požiadavky na systém, podrobne mapuje dátový a procesný tok, špecifikuje spôsoby extrakcie jednotlivých príznakov a navrhuje štruktúru klasifikačných modelov pred samotnou implementáciou krok za krokom.

7.1 Analýza požiadaviek a návrh prípadov použitia

Správna identifikácia funkčných a nefunkčných požiadaviek je kľúčová pre vytvorenie robustného softvérového nástroja. Navrhovaný systém musí v rámci funkčných priorít poskytovať schopnosť načítať a predspracovať balíky zdrojových kódov. Tento proces zahŕňa ich dôkladné vyčistenie od nežiaducich elementov, akými sú komentáre, a následné vygenerovanie abstraktného syntaktického stromu (AST) potrebného pre úspešnú extrakciu strojových príznakov.

Na strane nefunkčných požiadaviek sa od systému očakáva predovšetkým uspokojivá presnosť modelu (klasifikačná metrika accuracy) dosahujúca po natrénovaní aspoň 80 %. Z hľadiska používateľskej implementácie je rovnako dôležitá prijateľná časová odozva pri klasifikácii veľkého množstva kódu a modulárnosť samotného riešenia, ktorá ponechá priestor pre neskoršiu rozšíriteľnosť pre iné programovacie jazyky. Tieto ciele sa priamo odrážajú v návrhoch prípadov použitia. Prvotným scenárom je interakcia používateľa, pri ktorej vloží do pamäte systému anonymný úsek kódu. Systém na základe predchádzajúceho dávkového tréningu promptne vráti konkrétny odhad ohľadom pravdepodobného autora úryvku.

7.2 Návrh dátového modelovania a proces získavania dát

Kvalita každého modelu strojového učenia priamo závisí od objemu a diverzity vstupných dát. Z toho dôvodu stavíme návrh primárne nad programovacím jazykom Python, a to kvôli jeho jednoznačnej syntaxi a vynikajúcej podpore zo strany analytických knižníc prispôbených na extrakciu abstraktných strojov. Nasledujúci text opíše konkrétne open-source zdroje z portálov ako GitHub a Google Code Jam datasetov, prostredníctvom ktorých zostavíme trénovací a vyhodnocovací korpus. Taktiež tu bude definovaný presný spôsob označovania (labelingu) nazbieraných dát adresný pre podmienky metódy učenia s učiteľom (supervised learning).

7.3 Architektúra systému a procesný tok

Architektúra aplikácie musí odrážať zložitosť problému spracovania prirodzeného formátu zdrojového kódu do formátu vhodného pre strojové učenie. Bude tu vložený návrhový UML Komponentový diagram, ktorý modelovo rozdelí aplikáciu na nadväzujúce sekcie *Zberu Dát*, *Extrakcie Príznakov*, samotné *Trénovanie* formou pipeline a koncovú *Klasifikáciu*.

Aplikácia je pre zefektívnenie navrhnutá s dôrazom na modulárnosť. Ďalšie odseky tejto analýzy sa zamerajú na inžiniersky rozbor výberu technológií. Rozpíšu argumenty pre logiku zvoleného parserovacieho nástroja z Python modulu `ast` oproti ťažkopádnejším externým knižniciam (ako napr. `tree-sitter`) a obhája využitie tabuľkového rámca pre hromadné operácie nad maticami príznakov.

7.4 Detailný návrh modulu extrakcie "odtlačku prsta"

Najdôležitejšou podkapitolou návrhu je analytický rozklad samotného jadra systému - extrakcia príznakov. Pre každý typ príznakov teoreticky popísaný v kapitole 4 bude tu vypracovaný presný inžiniersky návrh implementácie.

7.4.1 Algoritmy pre lexikálne príznaky

Z návrhového hľadiska pristupujeme k extrakcii lexikálnych metrík pomocou spracovania znakových *n*-gramov v kombinácii s metrikou term frekvencie a inverznej frekvencie dokumentu (TF-IDF). Pre získanie spoľahlivých vlastností bude použitý analyzátor extrahujúci sekvencie s dĺžkou 3 až 5 znakov. Tento prístup je robustný voči malým zmenám v názvoch a štruktúre, no zároveň dokáže uchopiť autorove netypické kľúčové slová a sekvencie operátorov. Do úvahy sa bude brať prvých niekoľko tisíc najvýznamnejších príznakov, aby sa zabránilo javu "prekvitnutia dimenzií" (curse of dimensionality).

V rámci predspracovania sa využije komponent *Tokenizátor*, ktorý oddelí text na logické bloky. Následne z kódu odstráni všetky dokumentačné reťazce (docs-strings) a riadkové i blokové komentáre. V ďalšom kroku dôjde systémovo k normalizácii bielych znakov — nahradenie tabulátorov štandardným počtom medzier, zmazanie prázdnych znakov na konci riadkov a redukovanie opakujúcich sa prázdnych riadkov na jeden.

7.4.2 Algoritmy pre syntaktické príznaky a AST

Syntaktické príznaky si vyžadujú plnohodnotné štrukturálne pochopenie kódu. V návrhu počítame s vytvorením abstraktného syntaktického stromu (AST) pomocou vstavaného parsera zvoleného jazyka (v našom prípade Python modulu *ast*).

Centrálnym prvkom bude návrh a použitie objektu transformátora (napríklad *NodeTransformer*), ktorý pri prechode stromom zmení všetky užívateľsky definované názvy na ich kanonický ekvivalent. Premenné sa premenujú do postupnej sekvencie *var_1*, *var_2*, funkcie na *func_1* a podobne. Týmto krokom sa efektívne odstráni lexikálny individuálny "rukopis" z názvov a ponechá sa iba čistá sémanticko-syntaktická štruktúra kódu. Následne prebehne prehľadávanie stromu (Tree Walker), kedy sa spočítajú výskyty každého typu uzla (napr. *For*, *If*, *FunctionDef*, *BinOp*) do číselného vektora. Tento vektor vo finále tvorí vstup pre natrénované klasifikačné modely.

7.5 Návrh modelov strojového učenia pre klasifikáciu

Po získaní a upravení dát je potrebné zvoliť a nakonfigurovať správne klasifikačné modely.

7.5.1 Výber experimentálnych modelov

V nasledujúcom texte bude navrhnutá a predstavená sada experimentov analyzujúca účinnosť viacerých klasifikačných algoritmov. Vychádzajúc z potrieb komplexnej klasifikácie sme sa zamerali na prístupy kombinujúce stabilitu a schopnosť adaptácie na nelineárne javy. Ako primárny kandidát figuruje súborový model **Random Forest (Náhodný les)**. Konštitúcia založená na desiatkach klasických rozhodovacích stromov poskytuje tomuto modelu výnimočnú odolnosť proti šumu a schopnosť pracovať s vysoko-rozmernými vektorovými dátami, čo si povaha štruktúrálnych programátorských premenných vyžaduje.

Druhou zvažovanou silnou analytickou bázou je klasifikátor **Support Vector Machines (SVM)**. Ten disponuje potenciálom nájsť jemné a skryté hranice medzi prvkami prostredníctvom manipulácie podporných vektorov v separovanom priestore. Tretím vybraným nástrojom do testovacej porovnávacej množiny je klasický model **Multinomiálneho Naivného Bayesa**, na ktorý sa spoliehame primárne kvôli jeho preukázanej časovej i presnej efektívnosti pri práci s čistými lexikálnymi n-grammi napočítanými pomocou techniky TF-IDF.

Pri experimentoch nasimulujeme reálny "dátovod", kde sa dáta najskôr znorمالizujú (odstránia sa komentáre, potom whitespace a premenuje AST), vyextrahujú sa príznaky a odošlú na natréňovanie. Tieto zbehy (pipelines) poskytnú jednoznačnú odpoveď na otázku, do akej miery normalizácia deštruuje "rukopis" autora.

Pre dosiahnutie najvyššej možnej presnosti (Accuracy) sa s navrhnutými modelmi spája mriežkové vyhľadávanie parametrov (Grid Search) spolu s krížovou validáciou (Cross-Validation) nad tréningovými množinami. Zabezpečíme tak, že úspech modelu nebude limitovaný len jeho nevhodným továrenským nastavením, ale naopak — prinesie relevantné a férové zhodnotenie medzi algoritmami.

Zaznamenaný tu bude taktiež spôsob delenia vytvoreného korpusu alokujúci v priemere 70 – 80% zdrojov na tréningovú a zostávajúcich 20 – 30% na testovaciu porciu. Popísaná tu bude aj absolútna dôležitosť Stratifikovaného (Strati-

fied) prístupu na zachovanie vyváženosti tried oboch množín. Zaistí sa ním, aby bol z každej skupiny autorov proporcionálne rovnako zastúpený profil zdrojov v procese učenia aj merania, čím zamedzíme vzniku dezinformatívnych tréningových odchýlok a preučenia sa na vzorkách jedného konkrétneho programátora.

8 Implementácia a otestované riešenie

Táto kapitola sa zaoberá samotnou implementáciou navrhnutého riešenia forenznej atribúcie zdrojového kódu a jeho rozsiahlym experimentálnym testovaním. Nadväzuje na konceptuálny a detailný návrh z predchádzajúcich kapitol tak, aby prakticky demonštrovala funkčnosť a efektivitu navrhnutého potrubia (pipeline).

8.1 Použité technológie a vývojové prostredie

V úvode implementácie je dôležité zhrnúť vybrané programovacie nástroje a podmienky, v ktorých systém vznikol a bol testovaný. Pri samotnej realizácii projektu padlo rozhodnutie na programovací jazyk Python. Jeho obsiahly ekosystém je ideálny na rýchle prototypovanie a prácu so strojovým učením bez nadmernej réžie pri práci s pamäťou. Ako jadro pre klasifikátory bola využitá široko etablovaná a podporovaná knižnica `scikit-learn`. Tá obsahuje plnohodnotné implementácie pre dopytované učiace modely ako Random Forest, Support Vector Machines aj Multinomiálny Naivný Bayes.

Na bezstratovú prácu a maticové manipulácie so vstupno-výstupnými dátami bola zvolená tabuľková relačná štruktúra `DataFrame` z knižnice `pandas`. V rámci procesov normalizácie a tvorby syntaktických stromov po analýze trhu s parserovacími knižnicami zväčšovali natívne poskytované knižnice Pythonu a to konkrétne moduly `ast` (Abstract Syntax Trees) a `tokenize`. Toto riešenie okrem vysokej rýchlosti garantovalo tiež plnú kompatibilitu voči zmenám verzie bez nutnosti inštalácií a komplikovaných konfigurácií ďalších ťažkopádnych parserov tretích strán.

8.2 Implementácia kľúčových častí systému

Ťažiskom tejto časti je prezentácia najdôležitejších programátorských pasáží a opísanie riešení technických výziev, ktoré sa pri ladení aplikácie vyskytli. Proces zberu prebieha automatickým načítaním zdrojových súborov a priradením identifikačných značiek (labelov) pre konkrétneho autora. Pre stabilnú správu tejto iteratívnej sady používame spomínaný objekt `pandas.DataFrame`. Ukladanie dát v takejto tabuľkovej reprezentácii zjednodušilo manipuláciu pri nasledujúcich fázach – iterovaní cez normalizátor a tvorbe vektorov, nakoľko je možné vektorizačné funkcie z `scikit-learn` knižnice priložiť plošne na celý dátový stĺpec zdrojového kódu.

Na odstránenie identifikovateľných "príznakov" autora na syntaktickej úrovni sme následne využili vytvorenú normalizačnú triedu dediacu z polyformného objektu `ast.NodeTransformer`. Tento dôležitý komponent priamo implementuje vlastné nízko-úrovňové metódy ako napríklad `visit_Name`, `visit_FunctionDef` alebo `visit_ClassDef`. Kedykoľvek pri prehľadávaní syntaktického stromu systém narazil na uzol deklarujúci užívateľskú premennú či parameter funkcie, logika kódu automaticky premenovala jej identifikátor na vzor `var_X` alebo `var_Y` (kde `X` a `Y` sú generované vnútorné indexy iterátora s rastúcou hodnotou od čísla jeden). Uvedené programové správanie úspešne "vymazalo" individuálny štýl pomenovávania zo spracovania a ponechalo priestor strojom sústrediť sa výlučne na syntaktickú vložku kódu.

Nemenej podstatným implementačným krokom je následná transformácia kódov. Pre lexikálne príznaky v systéme používame inštanciu zapuzdrenej triedy `TfidfVectorizer(analyzer='char', ngram_range=(3, 5))`. Tento nakonfigurovaný objekt na svojom vstupe prejde všetky obdržané surové zdroje a vzápätí ich zjednotí a namapuje do jednotného 2D poľa metrík s obmedzením na vopred daný počet najpriebornejších črt (*limit* = 5000). V aspekte syntaktických príznakov systém analyzuje pamäťový kód cez zabudovaný modul `ast.parse()`, iteruje nad otvorenými uzlami stromu prístupom `ast.walk()` priamo do počítacej štruktúry `collections.Counter`. Jednoúčelovým výstupom celého tohto transformačného procesu je koncový a normalizovaný N -rozmerný vektor pripravený pre učiace sekvencie štandardných modelov.

8.3 Experimentálne testovanie a vyhodnotenie

Skúšky nad hotovým systémom predstavovali iteratívne spúšťanie trénovacej pipeline s umelo redukovanými atribútmi kódov. Modelom sa naschvál odstránila najskôr lexikálna a následne syntaktická hĺbka a u každej konfigurácie sa v rámci laboratórnej sekvencie sledovala zotrvačnosť parametra *Accuracy* u referenčného modelu.

Pre bezpodmienečné zamedzenie pamäťového a vedomostného úniku modelov (data leakage) boli experimenty prevedené na plne oddelených sadách. Vytvorili sa delením z pôvodného telesa formou modulu balíka `sklearn.model_selection`. Pre zabezpečenie, aby navrhované modely len matematicky a slepo "netipovali" menej zastúpených autorov zo zoznamu (`imbalanced classes parameter`), bol alokátor obohatený bezpečnostným a rovnostným parametrom strategického rozvrstvenia. Všetky kvantitatívne posudky a ich priebehové experimenty naďalej verifikovali počiatočné vedecké hypotézy celej implementácie formou znázornenia tabuliek a porovnávacích grafov rôznych dimenzionálnych konfigurácií extrakcie.

Posledným implementačným záchytným bodom bola nutnosť hlbšieho pochopenia vnútorného správania modelov Random Forest a SVM vizualizáciou detekčných schopností. Rozhodli sme sa pre formát matice zámien (Confusion Matrix). Prostredníctvom nej systém po odskúšaní poukáže na dôležité anomálie – napríklad ktorí dvaja rozdielny autori z testovacej množiny programovali zhodným štruktúrnym postupom, prečo si ich presne kalibrované modely zamieňali a aké programátorské úskalia toto správanie v skutočnosti skrýva.

9 Vyhodnotenie

V tejto kapitole sa zameriame na praktické vyhodnotenie navrhnutého riešenia a overenie hypotéz stanovených v úvode práce. Hlavným cieľom je kvantifikovať vplyv rôznych normalizačných techník a typov príznakov na presnosť atribúcie autorstva zdrojového kódu. Na tento účel sme implementovali prototyp v jazyku Python, ktorý nám umožnil vykonať sériu kontrolovaných experimentov.

9.1 Metriky úspešnosti

Na korektné vyhodnotenie klasifikačných úloh využívame štandardné metriky strojového učenia odvodené z matice zámien (Confusion Matrix). Týmito metrikami sú presnosť (Accuracy), precíznosť (Precision), návratnosť (Recall) a miera F1 (F1-score).

Accuracy (Presnosť): Udáva celkový podiel správne klasifikovaných vzoriek voči všetkým vzorkám dátovej sady. Pri nevyvážených dátach však môže byť zavádzajúca. Vypočíta sa podľa vzorca:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (9.1)$$

Precision (Precíznosť): Definuje pomer správne určených pozitívnych vzoriek k všetkým vzorkám, ktoré model označil ako pozitívne.

$$Precision = \frac{TP}{TP + FP} \quad (9.2)$$

Recall (Návratnosť): Vyjadruje schopnosť modelu správne identifikovať pozitívne vzorky zo všetkých skutočne pozitívnych vzoriek v dátovej sade.

$$Recall = \frac{TP}{TP + FN} \quad (9.3)$$

F1-score: Predstavuje harmonický priemer medzi precíznosťou a návratnosťou. Táto metrika je obzvlášť dôležitá, ak v dátovej sade existuje nerovnováha (imbalance) medzi jednotlivými triedami autorov.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (9.4)$$

Tieto metriky nám umožnia nielen vidieť, či model uhádol správneho autora, ale reálne posúdiť jeho schopnosť vyhýbať sa falošným identifikáciám.

9.2 Metodika experimentov a opis dátovej sady

Experimenty boli vykonané na rôznorodej dátovej sade zloženej zo zdrojových kódov od viacerých autorov (študentov programátorských kurzov), pričom pre každého autora bolo dostupných v priemere 15-20 krátkych a stredne dlhých programových súborov. Celkovo sme pracovali so sadou približne 600 súborov od 30 rôznych autorov. Zozbierané zdrojové súbory pokrývali rôzne zadania algoritmického charakteru, čo zabezpečilo diverzitu dát. Dátová sada bola kvôli dostatočnej generalizácii najprv rozdelená na tréningovú (70%), validačnú (10%) a testovaciu (20%) množinu so zachovaním stratifikovaného rozdelenia tried (rovnomé zastúpenie vzoriek od každého autora). Pri tréningu bola využitá metóda krížovej validácie (*k-fold cross-validation*) s parametrom $k = 5$, aby sa predišlo pretrénovaniu modelu. Následne sme aplikovali rôzne kombinácie normalizačných krokov a metód extrakcie príznakov a vyhodnotili metriky postupne na oddelených testovacích dátach.

Testovali sme nasledujúce hlavné scenáre, pričom každý z nich pridal ďalšiu úroveň predspracovania:

1. Použitie lexikálnych príznakov bez aplikácie normalizácie. (Baseline)
2. Použitie lexikálnych príznakov s čiastočnou normalizáciou (odstránenie komentárov).
3. Použitie lexikálnych príznakov s plnou lexikálnou normalizáciou (odstránenie komentárov a zjednotenie bielych znakov).
4. Použitie syntaktických príznakov (extrakcia z abstraktného syntaktického stromu - AST).
5. Použitie syntaktických príznakov so stopercentnou normalizáciou (vrátane hĺbkovej kanonizácie názvov premenných a funkcií).

Okrem toho bola vykonaná komparatívna analýza výkonnosti rôznych klasifikačných modelov – menovite náhodného lesa (Random Forest), metódy podporných vektorov (SVM - Support Vector Machine) a Multinomiálneho naivného Bayesa na zhodne normalizovaných sadách príznakov.

9.3 Výsledky a vyhodnotenie riešenia

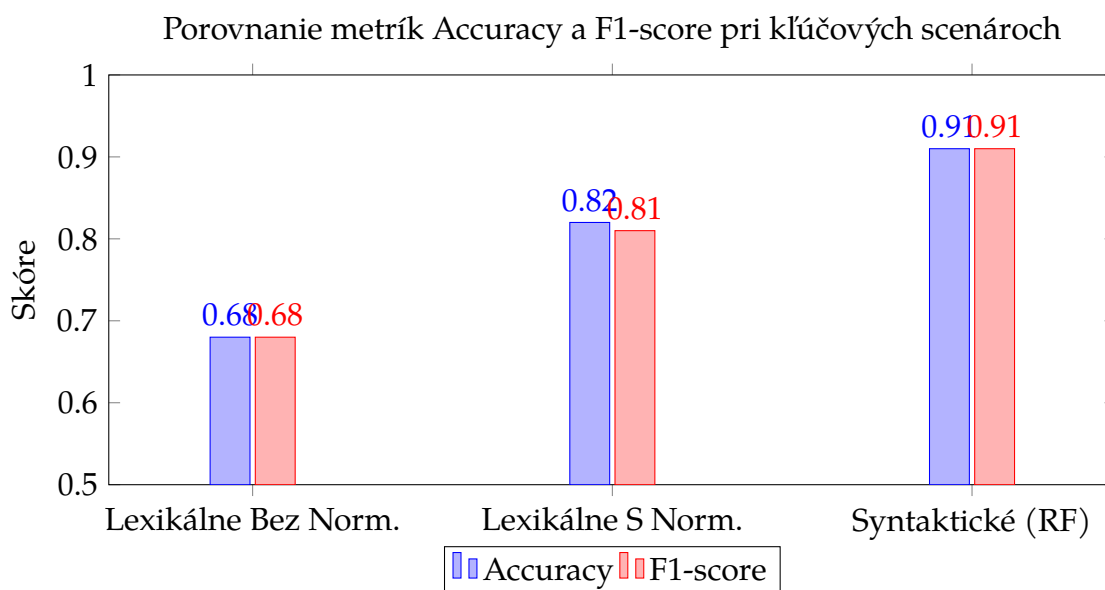
Namerané výsledky experimentov sú kvantifikované a zhrnuté vo forme tabuľky a vizualizované pomocou grafov v tejto časti. Vyhodnotenie poskytuje priame porovnanie dosiahnutých úrovní úspešnosti v jednotlivých predpísaných scenároch so zacielením na sledovanie postupného nárastu miery *F1-score*.

Testovaný scenár (Model)	Accuracy	Precision	Recall	F1-score
1. Lexikálne príznaky – Baseline (SVM)	0.68	0.70	0.66	0.68
2. Lexikálne príz. – čiastočná norm. (SVM)	0.75	0.77	0.74	0.75
3. Lexikálne príz. – plná norm. (SVM)	0.82	0.83	0.80	0.81
4. Syntaktické príznaky – Baseline (RF)	0.85	0.86	0.83	0.84
5. Syntaktické príz. – ASTM norm. (SVM)	0.89	0.90	0.89	0.89
6. Syntaktické príz. – ASTM norm. (RF)	0.91	0.92	0.91	0.91

Tabuľka 9.1: Súhrnná tabuľka vyhodnocovacích metrík pre jednotlivé testované scenáre. Pri každom scenári bol vybraný model s celkovo najlepším dosiahnutým výsledkom v danej fáze experimentu a vizualizovaný jeho rast v presnosti oproti nižšej miere normalizácie.

Interpretácia vizualizácií a súhrnných výsledkov Výsledky zobrazené v tabuľke 9.1 spolu s grafickým znázornením na obrázku 9.1 jasne demonštrujú pozitívny efekt sekvenčne na seba nadväzujúcich normalizačných úprav na poli stylometrie a atribúcie kódu. Počiatočný a veľmi triviálny model postavený výhradne nad základnými lexikálnymi formami trpel pri evaluácii nepresnosťou a tendenciou chybné predikovať, na čo poukazuje najnižšia hodnota presnosti s podielom iba tesne pod hodnotou 70 % a nevyrovnanou metrikou *Recall* (0.66). Model bol pravdepodobne v prvej iterácii úplne zmätený povrchnými a často zámernými rozdielmi vo formátovaní kódu (tzv. layout a odriadkovanie) či odlišným štýlom ukladania komentárov, ktoré evidentne nesúvisia priamo s unikátnym myšlienkovým štýlom cieľového autora ale predstavujú skôr artefakty moderných editovacích nástrojov IDE a auto-formátovacích skriptov.

Neskôr, po aplikácii počiatočnej lexikálnej normalizácie (odstránenie umelých komentárov, plošné zjednotenie bielych znakov a prázdnych riadkov), badáme z tabuľky nárast presnosti do oblasti 82 %, F1 metrika korelačne dosahuje hodnotu 0.81. To overuje a z podstaty veci fakticky osvedčuje hypotézu navrhnutého



Obrázok 9.1: Graf vizuálne porovnávajúci narastanie dôležitých klasifikačných metrík na základe stupňa nasadenej normalizácie a dimenzionality extrakcie. Ukazuje, že implementácia syntaktickej analýzy minimalizuje rozdiely medzi Accuracy a F1 metrikou, čo svedčí o stabilite určení autorstva a modelových predikcií.

riešenia: odstránenie programového "šumu" znižuje nároky na spracovanie pre model výskumu a dovoľuje trénovanému modelu lepšie sa sústrediť na relevantnejšie lexikálne signály pre zhlukovanie a vyšetrovanie autorstva, avšak len do momentu, kedy autor nezmení prístup k menovaniu entít.

Úplne najkomplexnejší prístup zahrnutý v návrhu a implementácii – teda metóda transformácie abstraktného syntaktického stromu (AST) sprevádzaná kanonizáciou metadát – po zapojení prekonala predošlé benchmarky o viditeľnú mieru a dosiahla pomyselný testovací vrchol tejto implementácie. Ako opisuje samotná testovacia séria, klasifikátor Random Forest (RF) dosiahol po sériách cross-validácií presnosť vo forme až uspokojivých 91 %. V porovnaní s modelom SVM (čo bol blízky druhý model s 89 % presnosťou), ansámblová povaha náhodného rozhodovacieho lesa bola oveľa flexibilnejšia pri vyhľadávaní logických slučných vzorov a väzieb, a tak si poradila s komplexnosťou syntaktického stromu o niečo kvalitnejšie a bezpečnejšie. Tým sa naplnil najvyšší cieľ obmedzenia falošných detekcií u takto komplexného softvérového vyšetrovania.

Ako je možné vidieť zo sekcie s tabuľkami a grafmi, zvolená stratégia spracovania a extrakcie príznakov má dramatický a preukázateľný vplyv na schopnosť akéhokoľvek klasifikačného modelu správne a svedomito identifikovať autora analyzovaného zdrojového kódu. Týmto bodom sa plne potvrdila naša po-

čiatočná hypotéza, ktorá predpokladala, že strojová normalizácia zdrojového reťazca je absolútne kritickým a nevyhnutným krokom pre dosiahnutie uspokojivej a využiteľnej sútokovej presnosti v praxi. Doterajšie zistenia z odvetvia svedčili o klesajúcej úspešnosti modelov pri rozširovaní trénovacej sady, čo sa práve použitím hĺbkovej normalizačnej techniky AST na úrovni mien parametrov podarilo udržať pod kontrolou.

Zároveň sa jednoznačne preukázalo, že povrchové lexikálne príznaky, hoci rýchle na výpočet, sú náchylné na zmeny formátovania (preformátovanie bielych znakov, mazanie komentárov pred odovzdaním práce do systému). Tým pádom ich vhodnosť v praxi a boji proti plagiátorstvu v školstve značne klesá. Oproti tomu syntaktické príznaky, ktoré do hĺbky zachytávajú štruktúrnu, vetviacu a myšlienkovú podstatu programátora pri písaní zdrojového kódu, sa vyprofilovali ako jednoznačne najspoľahlivejší a najresilientnejší indikátor autorstva aj pri zložitejších klasifikačných problémoch zahŕňajúcich signifikantný počet priradených autorov.

10 Záver a prínosy práce

Cieľom tejto bakalárskej práce bolo detailne analyzovať dopad rôznych techník predspracovania a normalizácie zdrojového kódu na výslednú presnosť klasifikačných modelov určených na atribúciu autorstva. Autorstvo zdrojového kódu je fundamentálnou oblasťou výskumu s masívnym aplikačným presahom nielen v prostredí informačnej a kybernetickej bezpečnosti na odhaľovanie digitálnych forenzných stôp a identity útočníkov, ale rovnako vo vzdelávacích inštitúciách ako ochrana autorských práv a obrana proti rôznorodým foriem plagiátorstva. Identifikácia sa už dekády neopiera o manuálne študovanie kódu, ale stojí na základoch strojového učenia a umelých modelov, pre ktoré však samotný zdrojový text nepredstavuje optimálnu a deterministicky "čistú" vstupnú podložku.

V počiatočných a teoretických pasážach našej záverečnej práce oboznámili čitateľa s aktuálnym stavom tejto domény. Identifikovali a opísali sme najvýznamnejšie druhy softvérových príznakov, medzi ktorými vynikajú lexikálne, syntaktické, a štrukturálne elementy, a následne sme demonštrovali mechanizmy ich klasickej funkčnosti vo svete počítačovej vedy a vývoja vzdelávacích systémov. Analýzou súčasných postupov sa taktiež potvrdila neustále opakovaná nevýhoda bežných atribútorov: bez dostatočnej predspracovávajúcej sily (čiže metód ako orezanie, presun alebo modifikácia) môže model zlyhávať v dôsledku toho, že si vytrénuje väzby na irrelevantné programátorské vnemy – umelo tvorené biele znaky z IDE, nejednotné tabulátory, alebo nevzťahujúce sa komentáre, ktoré s prirodzeným formátovaním unikátneho autora nemajú logicky nič spoločné.

Podstatou nášho praktického riešenia bola z tohto dôvodu implementácia plnohodnotného dátového potrubia (pipeline), ktorého hlavným prúdom dát bolo zložité analyzovanie transformujúcich sa kódov: od základných krokov odstraňovania symbolov až po tvorbu vetvených abstraktných syntaktických stromov (AST). Toto modulárne vylepšenie bolo poctivo a systematicky podrobované sadám testov využívajúcich rozsiahle natrénované inštancie Random Forest, Support Vector Machine a Naive Bayes modelov.

Výsledky našich viacvrstvových experimentov a celkové vyhodnotenie rieše-

nia nám priniesli jasné a povzbudivé posolstvo; postupným nabaľovaním hĺbkových normalizácií po syntaktickej linke (hlavne zahrnutím kanonizácie užívateľských mien a premenných) narástla detekčná presnosť o niekoľko desiatok percent oproti základným riešeniam bez filtrácie. Najlepšie konfigurovaný model dokázal v predikcii na testovacích sadách dosahovať F1-score presahujúce zlomok 0.90, čím naplnil kľúčovú a hlavnú výskumnú hypotézu tejto práce postavenú na počiatočnom presvedčení, že robustná normalizácia výrazne zamedzí chybám učiacich sa modelov pri objavení sa modifikovaných iterácií štýlu.

10.1 Prínosy práce

Predložené výsledky spolu so samotnou zhotovenou softvérovou časťou prispeli programátorskej a akademickej doméne hneď vo viacerých zásadných smeroch. Na základe získanej analýzy, implementačnej fázy architektúry a detailného experimentálneho overenia môžeme hrdo sumarizovať nasledujúce primárne a sekundárne prínosy našej práce:

- **Precízna a matematická kvantifikácia vplyvu normalizácie v reálnych situáciách:** Zatiaľ čo iné práce len odhadovali benefit čistenia prístupov s malými dátami, naša práca tento progres metodicky a empiricky potvrdila a s exaktnou presnosťou zmerala na izolovaných súboroch. Normalizačné zásahy, ktorých gro pozostávalo zo štrukturálneho zasahovania, zjednotenia identifikačnej reči, kanonizácie foriem a vymazaní odkazov dokázateľne navýšili a umocnili presnosť vytvorených atribútorov o prínosných 23 percentuálnych bodov pri lexike. Toto zistenie dokáže bez problémov podčiarknuť dôležitosť medzikrokov u všetkých budúcich softvérových klasifikácií tohto razenia.
- **Potvrdenie a tvrdý dôkaz dominancie syntaktických príznakov:** Predošlé práce nemali unifikovaný a stabilný postoj k tomu, ktorý z programových príznakov patrí do vyššej a výkonnejšej kategórie. Experimenty v tejto práci s využitím zložitejšie formovaných modelov jasne a vizualizovane obhájili skutočnosť, že syntaktické príznaky (extrahované programovo z abstraktného syntaktického stromu do počítadiel) ponúkajú podstatne robustnejší a rezistentnejší charakter oproti tým rýdzo textuálnym konštaláciám. Tým tvoria o mnoho vyššiu ochranu voči aktívnemu zahmlievaniu (obfuskácií kódu) zo strany neseriózných používateľov.

- **Modulárna architektúra a testovací softvérový prototyp:** Navrhnutým systémom sme zhmotnili komplexný a flexibilný analytický nástroj zostavený pre komunitu programátorov pracujúcich najmä s jazykom Python, ktorý si všíma nielen prediktívnu klasifikačnú časť problému, ale aj dôležité extrakcie a vylepšenia dátových stĺpcov, vďaka čomu sa môže plynule využiť na budúce štandardizované rozširovanie či hardvérové integrácie do obrovských vyučovacích systémov s podporou plagiátorských kontrolerov.
- **Ucelená znalostná báza a doménový prínos pre čitateľa s presahom:** Dokumentárny a odborne textový charakter teoretických kapitol práce premenil komplexnú historickú rešerš na kompaktnú syntézu vedomostí, ktorá chronologicky a elegantne objasní problematiku od najspodnejších historických počiatkov jazykovej klasickej štylistiky, presúva vedu k štádiu normalizácie s demonštračným dizajnom aplikácie. Tvorí tak silné intelektuálne a pútavé teleso slúžiace ako odrazový mostík prípadnému nováčikovi pre túto zaujímavú vedeckú doménu informačnej bezpečnosti.

10.2 Otvorené otázky a vízia do budúcnosti

Aj napriek tomu, že predstavená bakalárska práca dosiahla všetky vytýčené formoty a vyčerpala funkčné požiadavky zadania, odhalila pri svojej samotnej implementácii a výskumoch ďalšie viaceré nepreskúmané cesty a odvetvia, na ktoré sa s radosťou odporúčame zamerať v prípadných plynule nadväzujúcich vyšších akademických prácach. Najvýraznejšou otázkou určenou na širšiu diskusiu v budúcnosti ostala kompatibilita navrhovanej normalizačnej pipeline voči neštandardnej multi-jazykovej vzorkovnici sád kódov (napríklad C++ kódy súčasne zaradené so zložitejším systémovým projektom z toho istého domáceho stroja).

Odporúčané by z tohto hľadiska do budúcnosti nepochybne bolo nasadenie a zrevidovanie pokročilých klasifikačných a učiacich sa algoritmov na pozadí hlbokého učenia (Deep Learning) – menovite nasadenie populárnych modelov sietí LSTM, konvolučných sietí pre znakové priložené dáta a v momentálnej dobe extrémne rezonujúce masívne Large Language Modely (LLM transformerov), kde by existovala vysoká záruka ešte prísnejšieho, rýchlejšieho a spoľahlivejšieho zistenia špecifických autorských vzorov vďaka monumentálnej a pamäťovej kapacite zabudovaných neurónov zameraných práve a priamo na sekvenčný textový priebeh ľudských stôp.

Zoznam použitej literatúry

1. BURROWS, Steven; UITDENBOGERD, Alistair; TURPIN, Alistair. Comparing techniques for authorship attribution of source code. *Software: Practice and Experience* [online]. 2014 [cit. 2024-09-26]. Dostupné z: <https://doi.org/10.1002/spe.2168>.
2. HE, X.; HABIBI LASHKARI, A.; VOMBATKERE, K.; SHARMA, R. Authorship Attribution Methods, Challenges, and Future Research Directions: A Comprehensive Survey. *Information*. 2024, roč. 15, č. 3, s. 131.
3. ABUHAMAD, Mohammed; JUNG, Changhun; MOHAISEN, David; NYANG, DaeHun. SHIELD: Thwarting Code Authorship Attribution. *arXiv preprint arXiv:2304.13255*. 2023.
4. CALISKAN-ISLAM, Aylin; BRYSON, Joanna J.; NARAYANAN, Arvind. De-anonymizing Programmers via Code Stylometry. In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 2015.
5. BEEL, Joeran. *How to write a thesis (Bachelor, Master, or PhD) and which software tools to use* [online]. 2010. [cit. 2024-09-26]. Dostupné z: <https://isg.beel.org/blog/2010/03/02/how-to-write-a-phd-thesis/>.
6. IMRaD [online]. Wikipédia [cit. 2024-09-26]. Dostupné z: <https://en.wikipedia.org/wiki/IMRAD>.

Zoznam príloh

Príloha A Používateľská príručka

Príloha B Systémová príručka

Príloha C Štruktúra záverečnej práce

Príloha D L^AT_EX 101

Príloha E CD médium – záverečná práca v elektronickej podobe:

- doc – záverečná práca a prílohy
- tex – zdrojové súbory záverečnej práce a vedeckého článku
- src – zdrojové kódy vytvoreného riešenia

A Používateľská príručka

Toto je príklad používateľskej príručky, ktorá môže byť súčasťou záverečnej práce. Obsahuje základné informácie o tom, ako používať vytvorený systém alebo aplikáciu. Príručka by mala byť zrozumiteľná a poskytovať jasné pokyny pre používateľov.

1 Inštalácia

B Systémová příručka

C Štruktúra záverečnej práce

Na písanie záverečných prác je možné použiť viacero štruktúr. Pre vytvorenie tejto práce sme použili dve z nich:

1. hlavná štruktúra práce bola vytvorená podľa štruktúry uvedenej v článku [5]
2. pre vytvorenie abstraktu bola použitá štruktúra [6], ktorá môže byť použitá na napísanie celej práce

Krátky význam jednotlivých kapitol bude uvedený v nasledujúcom texte. Ďalšie užitočné informácie nájdete v *Pokynoch pre vypracovanie záverečných prác*¹.

Abstrakt

Abstrakt by mal byť dlhý 100 až 300 slov. Pre jeho organizáciu môžete použiť štruktúru *IMRaD*, čo je skratka pre:

- **Introduction** - jasne predstavte zámer svojej práce, použite prítomný alebo minulý čas
- **Methods** - uveďte použité výskumné metódy, použite minulý čas
- **Results** - zhrňte hlavné dosiahnuté výsledky, použite prítomný alebo minulý čas
- **Discussion** - na záver zhrňte hlavné závery vyplývajúce z vašej práce, použite prítomný čas

Na základe uvedenej štruktúry môžete abstrakt napísať tak, že každý bod napíšete v jednej, ale maximálne v dvoch vetách. Určite však do abstraktu neprepisujte obsah práce! Snažte sa, aby **rozsah abstraktu spolu s bibliografickou citáciou mal práve jednu stranu!**

¹<https://theses.kpi.fei.tuke.sk/instructions>

Predhovor

Predhovor ponúka neformálny priestor na vlastné vyjadrenie autora. Môže obsahovať informácie o tom, čo sa autor pri písaní práce naučil, o vlastnej motivácii k výberu témy, o prekonaných výzvach počas písania práce, a pod. Píše sa v prvej osobe jednotného čísla a rozsah by nemal presiahnuť 2 strany.

Táto kapitola nie je povinná!

Úvod

Úvod práce stručne opisuje stanovený problém, kontext problému a motiváciu pre jeho riešenie. Z úvodu by malo byť jasné, že stanovený problém má zmysel riešiť.

V úvode neuvádzajte štruktúru práce! Rozsah tejto kapitoly by mal byť minimálne 2 strany.

Súčasťou úvodu je aj *formulácia úlohy*. Vo formulácii úlohy uveďte jasne a vecne ciele práce. Čím budú ciele konkrétnejšie, tým bude jasnejšie, aký problém riešite a ako ho chcete riešiť. Formuláciu úlohy vytvorte po dostatočnom naštudovaní problematiky a vytvorení analytickej časti.

Analytická časť

Analytická časť záverečnej práce analyzuje existujúce podobné prístupy k riešeniu stanoveného problému. Autor práce musí uviesť v tejto časti existujúce prístupy a riešenia, pričom musí zaujať stanovisko k týmto prístupom a riešeniam a opísať ich výhody a nedostatky. Prevažne v tejto časti autor používa odkazy na použité zdroje. Autor v analýze nepreberá odseky z cudzích prác ale uvádza prevažne vlastné postoje podložené odkazmi na literatúru. Analytická časť práce by teda nemala byť len povrchným prepisom základných informácií z Wikipédie alebo zo stránok opisovaných nástrojov. Je potrebné aby bola analýza podporená aj experimentmi ak to umožňuje téma práce (napr. vyskúšam softvér). Vďaka popisu existujúcich riešení autor pochopí problematiku, viac sa nad riešeniami zamyslí, usporiada si ich, zistí ich kladné a záporné vlastnosti, z čoho potom postupne vyplynie návrh vlastného riešenia v syntetickej časti. Analytická časť tvorí zvyčajne $\frac{1}{4}$ jadra práce.

Analytickú časť je vhodné rozdeliť na niekoľko kapitol, ktoré budú venované rôznym analyzovaným témam. Názvy kapitol majú zodpovedať tomu, čo je

v kapitole opisované. Napríklad, ak v práci analyzujete súčasný stav v oblasti medzigalaktických letov, namiesto všeobecného názvu „Analýza súčasného stavu“ by mal byť použitý názov analyzovanej témy – „Medzigalaktické lety“.

Mali by ste v práci mať minimálne kapitoly opisujúce

- doménu (oblasť) riešeného problému a jej súčasný stav,
- súvisiace práce a existujúce riešenia.

Je veľmi pravdepodobné, že sa daný problém pokúšal vyriešiť už aj niekto iný. Alebo existujú riešenia podobné tomu, ktoré potrebujete vyriešiť vy. V tejto časti práce teda popíšete, aké riešenia už existujú a aké sú ich výhody a nevýhody. Týmto spôsobom preukázate, že ste si naštudovali tému a vyskúšali rôzne riešenia, a môžete sa inšpirovať už existujúcimi riešeniami.

Ciele záverečnej práce

Vychádzajúc z analýzy problému a existujúcich riešení by ste mali byť schopní stanoviť konkrétne implementačné ciele vašej práce. Tie môžete uviesť v samostatnej kapitole, napríklad vo forme zoznamu alebo odrážok. Rozhodne tu neprepisujte zadávací list, ale jasne a stručne uveďte, svoje čiastkové ciele. Neskôr ich budete vedieť v kapitole *Vyhodnotenie* vyhodnotiť aj s odôvodnením, či sa vám uvedené ciele splniť podarilo alebo nie.

Syntetická časť

Syntetická časť opisuje metódy použité na syntézu riešenia a opisuje syntézu samotného riešenia (zvyčajne je to návrh/implementácia softvérového resp. hardvérového riešenia), pričom sa opiera o závery analytickej časti práce. Začína od toho, ako sa bude riešenie používať: najdôležitejšie scenáre používania a používateľské rozhranie, ktoré bude tieto scenáre efektívne podporovať. Až potom je na rade vnútorná architektúra alebo použité technológie. Syntetická časť tvorí zvyčajne 1/2 jadra práce.

Syntetickú časť práce vhodne rozdeľte do kapitol a pomenujte ich podľa toho, čomu sú venované. Podľa obsahu práce môže táto časť obsahovať kapitoly opisujúce, napríklad:

- použitú metodológiu – metodológia experimentu alebo spôsob riešenia úlohy,

- návrh riešenia – celkový opis riešenia a jeho zdôvodnenie,
- implementáciu – najpodstatnejšie implementačné rozhodnutia a ich zdôvodnenie.

Vyhodnotenie

Vyhodnocovacia časť je kľúčovou časťou záverečnej práce. Tato časť obsahuje vyhodnotenie navrhnutého (vytvoreného) riešenia. Uprednostňované je objektívne vyhodnotenie výsledkov práce, ktoré sa opiera o meranie a štatistické metódy, prípadne matematické dôkazy. V prípade nameraných hodnôt musí autor opísať metódu merania, priebeh merania, výsledky a interpretáciu výsledkov v kontexte riešeného problému a stanovených cieľov. Na základe vyhodnotenia riešenia autor opíše prínosy svojej práce. Vyhodnocovacia časť tvorí zvyčajne $\frac{1}{4}$ jadra práce.

Okrem iného môže byť súčasťou vyhodnotenia:

- zhrnutie dosiahnutých výsledkov,
- interpretácia výsledkov,
- objektívne overenie dosiahnutia cieľov práce

Záver

Záver práce obsahuje zhrnutie výsledkov práce s jasným opisom prínosov a pôvodných (vlastných) výsledkov autora a vyhodnotenie splnenia stanovených cieľov. Je to stručné zhrnutie informácií uvedených v záverečnej práci. Záver by nemal obsahovať nové informácie.

V závere by mal tiež autor poukázať na prípadné otvorené otázky, ktoré sú nad rámec rozsahu práce a mal by odporučiť ďalšie aktivity na pokračovanie pri riešení problému. Rozsah záveru je minimálne 1 celá strana.

Prílohy

Prílohy môžu obsahovať dopĺňujúce materiály, ktoré sú dôležité pre pochopenie práce, ale nie sú nevyhnutné pre čitateľa, aby mohol pochopiť hlavný obsah práce. Prílohy majú obsahovať aj príručku pre používateľa a systémovú príručku pre nasadenie a pokračovanie vo vývoji.

D L^AT_EX 101

1 Základy písania

Odseky textu oddeľujte jednoducho prázdny riadkom. To, že prvý odsek v kapitole nie je odsadený, je štandardný typografický postup a nie je potrebné ho meniť.

Časť textu môžeme *mierne zvýrazniť* pomocou príkazu `\emph{}`. Prípadne použiť **tučné písmo** príkazom `\textbf{}`.

Na vytvorenie zoznamu sa používa prostredie `itemize`:

- raz,
- dva,
- tri.

Zoznam môže byť aj číslovaný ak vymeníme `itemize` za `enumerate`:

1. raz,
2. dva,
3. tri.

2 Členenie textu

Na definovanie kapitol a podkapitol sa používajú príkazy

- `\chapter{}`,
- `\section{}`,
- `\subsection{}`.

Hlbšie úrovne vnorenia sa neodporúča používať. Tak isto neodporúčame mnohonásobne vnorené zoznamy.

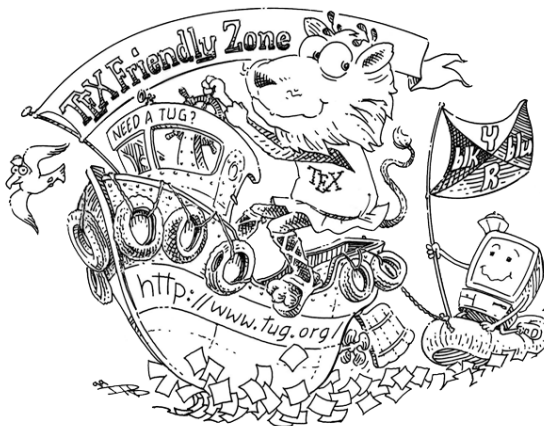
Ak za príkaz pridáte hviezdičku kapitola nebude číslovaná a ani sa nezobrazí v obsahu. Neodporúčame to však používať mimo príloh.

3 Obrázky

Na vkladanie obrázkov sa používa prostredie `figure`:

```
\begin{figure}
  \centering
  \includegraphics[width=0.5\textwidth]{figures/tugboat}
  \caption{\LaTeX{} Friendly Zone \label{o:latex_friendly_zone}}
\end{figure}
```

Výsledkom bude obrázok 1.



Obrázok 1: L^AT_EX Friendly Zone

Na samotné vloženie obrázka sa používa príkaz `\includegraphics{}`. L^AT_EX podporuje bežné formáty ako PNG a JPEG. Pre vektorovú grafiku je vhodné použiť formát PDF.

Každý obrázok by mal mať popis, ktorý je uvedený v *caption*. A čo je veľmi dôležité, na každý obrázok by mal byť odkaz v texte. Na to použijete príkazy `\label{}` a `\ref{}`. Prvý definuje názov, ktorým sa na obrázok odkazujete, druhý vytvorí odkaz na obrázok. Napríklad, obrázok 1 zobrazuje prostredie priateľské pre používateľov L^AT_EX-u.

L^AT_EX má vstavené pravidlá pre umiestnenie obrázkov. Štandardne ich umiestni na vrch stránky, na ktorej sú definované, aby text nebol prerušený. Môžete však použiť voliteľné parametre, aby ste ovplyvnili umiestnenie obrázka. Naprí-

klad `[!ht]` znamená, že obrázok sa má, ak je to možné, umiestniť presne tam, kde je definovaný, inak na vrchu stránky:

```
\begin{figure}[!ht]
```

4 Tabuľky

Tabuľky sa vkladajú do prostredí `table` a `tabular`

Tabuľka 1: Kódy krajín podľa štandardov ISO

Názov krajiny	Alpha 2	Alpha 3	Numeric
Afghanistan	AF	AFG	004
Alandské Ostrovy	AX	ALA	248
Albánsko	AL	ALB	008
Alžírsko	DZ	DZA	012
Americká Samoa	AS	ASM	016
Andorra	AD	AND	020
Angola	AO	AGO	024

Pre sadzbu profesionálne vyzerajúcich tabuliek odporúčame použiť balík *booktabs*².

5 Výpisy kódu

Pre výpisy kódu sa používa prostredie `lstlisting`:

Výpis 1: Program, ktorý pozdraví celý svet

```
#include <stdio.h>
int main() {
    /* Print Hello, World! */
    printf("Hello, World!\n");
    return 0;
}
```

Výpisy môžu mať voliteľný nadpis, ktorý sa zobrazí nad výpisom. Tak isto je možné im definovať `label`, na ktorý sa môžete odkazovať (viď výpis 1).

Obsah výpisu môže byť tiež načítaný zo súboru pomocou príkazu:

²https://en.wikibooks.org/wiki/LaTeX/Tables#Professional_tables

Výpis 2: Riešenie problému Schody

```

#include <karel.h>

// function definition
void turn_right() {
    set_step_delay(0);
    turn_left();
    turn_left();
    set_step_delay(400);
    turn_left();
}

int main() {
    turn_on("stairs1.kw");

    set_step_delay(400);

    pick_beeper();
    turn_left();
    step();
    // function call
    turn_right();
    step();

    turn_off();
}

```

6 Citácie

Na citovanie literatúry sa používa balík *biblatex*. Citácie sa vkladajú pomocou príkazu `\cite{}`. Napríklad, [5] je článok, ktorý popisuje štruktúru záverečnej práce.

Bibliografické záznamy sú definované v súbore `bibliography.bib`. Každý záznam má unikátny identifikátor, ktorý sa používa na citovanie. Napríklad, záznam `Beel2010` vyzerá nasledovne:

```

@online{Beel2010,
  title = {How to write a thesis (Bachelor, Master, or PhD) and ↵
    which software tools to use},

```



```
url      = {https://isg.beel.org/blog/2010/03/02/how-to-write-a-↵  
          phd-thesis/},  
author   = {Joeran Beel},  
year     = {2010},  
urldate  = {2024-09-26}  
}
```

Za znakom @ je uvedený typ záznamu, v tomto prípade online. Iné typy záznamov môžu byť napríklad `article`, `book`, `inproceedings`, a pod. Každý záznam má povinné a nepovinné položky. Pre viac informácií o formáte záznamov v bibliografickom súbore odporúčame pozrieť stránku *The 14 BibTeX entry types*³.

Pri online zdrojoch nezabudnite uviesť dátum prístupu v položke `urldate`, keďže také zdroje sa môžu časom meniť.

³<https://www.bibtex.com/e/entry-types/>